

計算機科学応用演習

final exam レポート

東京理科大学 創域理工学部 情報計算科学科 3年

学籍番号 6324034

木村 竜輝

提出日: 2026年7月2日

1 課題

以下の 8 つの課題を満たす Python プログラムを作成し、実行結果を確認する。

1. **動画の読み込み:** 動画 (People - 84973.mp4) を読み込み、動画の縦・横のサイズ、FPS、総フレーム数を出力する。
2. **サムネイル画像の作成:** サムネイルとして動画 (People - 84973.mp4) 中のフレーム画像を 1 枚出力し、保存する。ただし、サムネイルは単に 1 フレーム目ではなく、中央のフレーム、または自分で良いと思った選択方法で選んだ代表的なフレームとする。
3. **ファイル名のパース:** 動画のファイル名 (People - 84973.mp4) の文字列を処理して、タイトル (People) と ID(84973) に分けるプログラムを作成し、その結果を出力する。他の動画 (Cat - 66004.mp4 など) でも同様のアルゴリズムで分けられるようにする。
4. **色反転と動画の保存:** 動画 (Cat - 66004.mp4) の各フレームの色を反転した動画を作成し、保存する。反転後の画素値は「画素値の最大値 - 元の画素値」とする。縦・横のサイズは元のサイズからそれぞれ 0.3 倍する。保存するファイル名は 6324034_cat.reverse.mp4 とする。cv2.bitwise_not() は使用しない。
5. **音声の可視化:** 音声 (report_audio.wav) には時間-周波数領域にメッセージが隠されている。パラメータを適宜調節して解析し、可視化したグラフを描画する。
6. **n-gram:** テキスト (text_english.txt) に出現する 2-gram の出現頻度を調べ、上位 10 個をグラフ化する (6-a)。さらに stop words を設計・除去したうえで上位 10 個の 2-gram をグラフ化し、除去しない場合と比較する (6-b)。
7. **動画の変わり目検出:** 4 つの動画を結合した動画 (Detection.mp4) から、グレースケール化した隣接フレーム間の画素値差の総和 (変化量) を用いて変わり目 (3 箇所) を自動検出する。横軸フレーム・縦軸変化量のグラフを描画し (7-a)、閾値の決め方を 2 種類以上試して変わり目フレームを出力する (7-b)。absdiff() などの関数は使用しない (numpy は使用可)。
8. **オブジェクト指向:** 動画のパスを受け取ってインスタンス化する Video クラスを作成する。インスタンス変数 (パス・タイトル・ID・サムネイル)、課題 2・3・4 の処理のメソッド化、インスタンス変数を辞書で返す get_info()、およびフレーム処理関数 (二値化・グレースケール化・色反転) を引数として受け取る高階関数方式の動画処理メソッドを実装し、動作を確認する。

2 アルゴリズムの説明

本節では、各課題を解くために採用したアルゴリズムを、プログラミング言語に依存しない形で説明する。

2.1 動画の読み込み (課題 1)

動画ファイルは、連続する静止画 (フレーム) の列と、その再生速度などのメタデータから構成される。動画コンテナには幅・高さ・1 秒あたりのフレーム数 (FPS)・総フレーム数がメタデータとして格納されているため、動画をデコーダで開き、これらのメタデータを問い合わせ出力すればよい。

2.2 サムネイル画像の作成 (課題 2)

サムネイルは動画の内容を 1 枚で代表する画像である。先頭フレームはフェードインや無内容な場面であることが多く、代表性が保証されない。そこで本課題では「動画全体の明るさ分布に最も近い明るさ分布を持つフレームが、動画を最もよく代表する」という基準を採用した。手順は以下のとおりである。

1. 動画から一定間隔でフレームをサンプリングする。
2. 各フレームをグレースケール化し、画素値の正規化ヒストグラム h_i を計算する。
3. 全サンプルの平均ヒストグラム $\bar{h}$ を求める。
4. 各フレームについて平均ヒストグラムとのユークリッド距離を計算する。

$$d_i = \|h_i - \bar{h}\|_2 \quad (1)$$

5. d_i が最小のフレームを代表フレームとして選択する。

式 1 の d_i が小さいフレームは動画の平均的な画面構成に近く、場面転換の途中や極端に明るい・暗い例外的なフレームが選ばれることを避けられる。

2.3 ファイル名のパース (課題 3)

ファイル名は「タイトル - ID. 拡張子」という規則で構成されている。したがって、(1) まず拡張子を取り除き、(2) 残った文字列を区切り文字列「-」(空白・ハイフン・空白) で分割すれば、前半がタイトル、後半が ID となる。タイトル自体にハイフンが含まれる場合を考慮し、分割は右端の区切りで 1 回だけ行う。これにより「A-B - 12345.mp4」のようなファイル名でもタイトル「A-B」と ID「12345」に正しく分けられる。

2.4 色反転と動画の保存 (課題 4)

画素値が 8 ビット (最大値 255) で表されるとき、色反転は各画素・各チャンネルの値 p を次式で置き換える操作である。

$$p' = 255 - p \quad (2)$$

式 2 により、暗い画素 (p が小さい) は明るく (p' が大きく)、明るい画素は暗く変換される。動画に対しては、全フレームに式 2 を適用し、縦・横を 0.3 倍に縮小したうえで、元動画と同じ FPS で動画ファイルとして書き出す。

2.5 音声の可視化 (課題 5)

音声信号は時間領域の 1 次元信号であり、そのままでは周波数方向に埋め込まれたメッセージを読み取れない。そこで短時間フーリエ変換 (STFT: Short-Time Fourier Transform) を用いる。STFT は信号を長さ N の窓で切り出しながら、窓ごとに離散フーリエ変換を行う操作であり、信号 $x[n]$ 、窓関数 $w[n]$ に対して次式で定義される。

$$X(m, k) = \sum_{n=0}^{N-1} x[n + mH] w[n] e^{-j2\pi kn/N} \quad (3)$$

ここで m は窓の位置 (時間方向)、 k は周波数ビン、 H はホップ幅 (窓をずらす幅) である。 $|X(m, k)|^2$ を時間-周波数平面に濃淡で描いたものがスペクトログラムであり、周波数方向に描かれた文字列を画像として可視化できる。窓長 N を大きくすると周波数分解能が上がる代わりに時間分解能が下がる (不確定性関係) ため、メッセージが読みやすくなるよう N とホップ幅を調節する。また、パワーは対数 (dB) スケール

$$P_{\text{dB}} = 10 \log_{10} |X(m, k)|^2 \quad (4)$$

で表示することで、振幅差の大きい成分が共存していても濃淡の判別ができるようにする。

2.6 n-gram(課題 6)

2.6.1 2-gram の抽出と頻度集計 (6-a)

2-gram とは、文章を単語単位で区切ったときの連続する 2 語の組である。単語列 $w_1, w_2, \dots, w_M$ から $M - 1$ 個の 2-gram $(w_1, w_2), (w_2, w_3), \dots, (w_{M-1}, w_M)$ が得られる。テキストを (1) 小文字化し、(2) アルファベット列のみを単語として切り出し、(3) 隣接する 2 語の組を列挙し、(4) 組ごとの出現回数を数え、(5) 出現回数の降順に並べて上位 10 個を棒グラフとして描画する。

2.6.2 stop words 除去 (6-b)

stop words は冠詞・be 動詞・前置詞・接続詞・代名詞など、出現頻度は高いが文章の内容的特徴を担わない語である。単語列から stop words を取り除いた後に 2-gram を構成することで、内容語同士の接続だけが集計対象となり、文章の主題を反映した 2-gram が上位に現れることが期待される。除去は「単語列をフィルタしてから 2-gram を作る」方式とした。このため、元の文章では stop words を挟んで離れていた 2 つの内容語が新たに隣接する 2-gram として数えられることがある。この設計の影響は考察で述べる。

2.7 動画の変わり目検出 (課題 7)

動画の変わり目 (ショット境界) では、隣接フレーム間で画面全体の画素値が大きく変化する。そこで、フレーム t のグレースケール画像を $I_t(x, y)$ とし、隣接フレーム間の変化量 $D(t)$ を次式で定義する。

$$D(t) = \sum_x \sum_y |I_t(x, y) - I_{t-1}(x, y)| \quad (5)$$

同一ショット内ではフレーム間の変化は被写体やカメラの動きによる小さな値に留まるが、ショット境界では画面全体が入れ替わるため $D(t)$ が突出して大きくなる。したがって $D(t)$ がある閾値 θ を超えるフレームを変わり目と判定できる。閾値の決め方として次の 3 方式を用いた。

1. 統計量に基づく方式: $D(t)$ の平均 μ と標準偏差 σ から

$$\theta = \mu + k\sigma \quad (6)$$

とする。変わり目は全フレームのごく一部なので、 $D(t)$ の分布はショット内変化を中心に集中し、変わり目は外れ値として $\mu + k\sigma$ の外側に現れる。

2. 上位 n ピーク方式: 変わり目の個数 (本課題では 3) が既知であることを利用し、 $D(t)$ の大きい順に n 個のフレームを選ぶ。
3. 移動平均基準方式: $D(t)$ の近傍 W フレームの移動平均 $\bar{D}(t)$ を局所的な基準線とし、 $D(t) > k'\bar{D}(t)$ となるフレームを選ぶ。動きの多い区間と静かな区間が混在しても、局所的な変化の水準に応じて閾値が自動的に追従する。

2.8 オブジェクト指向設計 (課題 8)

課題 2・3・4 の処理は、いずれも「1 本の動画」に付随する操作である。そこで動画 1 本を表す Video クラスを定義し、動画のパス・タイトル・ID・サムネイルをインスタンス変数として保持し、各処理をメソッドとして所属させる。これにより、データ (動画の属性) とそれを操作する処理が 1 つの単位にまとまり、複数の動画を扱う場合も動画ごとにインスタンスを作るだけでよくなる。

さらに、動画処理を「フレーム 1 枚を変換する処理」と「全フレームへの適用と入出力」に分離する。フレーム変換関数 (色反転・グレースケール化・二値化) を引数として受け取る高階関数方式にすることで、新しい画像処理を追加するときも動画入出力のコードを複製せず、フレーム変換関数を 1 つ書いて渡すだけで済む。

3 プログラムの説明

プログラムは Python で作成した。使用した主なライブラリは OpenCV (cv2)、numpy、matplotlib、scipy である。プログラム全体は付録 A に示す。本節では、前節のアルゴリズムをどのように実装したかを、関数ごとに説明する。

3.1 パス設定

課題指定の Colab 上のディレクトリパスを既定値とし、環境変数 DATASET_ROOT が設定されている場合のみ、その値をルートとして各ディレクトリパスを組み立てる。Colab 上では環境変数を設定しないため指定どおりのパスとなり、手元の計算機では環境変数を設定するだけで同じプログラムを動作確認できる。

3.2 課題 1: task1_video_info

入力は動画ファイルのパス、出力は幅・高さ・FPS・総フレーム数である。cv2.VideoCapture で動画を開き、get メソッドに CAP_PROP_FRAME_WIDTH、CAP_PROP_FRAME_HEIGHT、CAP_PROP_FPS、CAP_PROP_FRAME_COUNT を渡して各メタデータを取得し、表示する。幅・高さ・総フレーム数は本来整数なので int に変換して表示する。

3.3 課題 2: task2_thumbnail と compute_gray_histogram

compute_gray_histogram は、フレームを cv2.cvtColor でグレースケール化し、np.histogram で 64 ビンのヒストグラムを計算して、総和が 1 になるよう正規化して返す。task2_thumbnail は動画を先頭から読みながら sample_step フレームに 1 枚の間隔でヒストグラムを蓄積し、平均ヒストグラムとの L2 距離 (式 1) を np.linalg.norm で計算して、距離最小のフレーム番号を求める。その後 cap.set(cv2.CAP_PROP_POS_FRAMES, best_index) で該当フレームへ移動して読み直し、cv2.imwrite で JPEG として保存する。

3.4 課題 3: task3_parse_filename

入力はファイル名の文字列、出力はタイトルと ID の組である。os.path.splitext で拡張子を除き、rsplit("-", 1) で右端の区切りから 1 回だけ分割する。分割結果が 2 要素でない場合は想定外の形式として ValueError を送出する。前後の余分な空白は strip で除去する。

3.5 課題 4: task4.reverse_video

入力動画パス・出力パス・縮小率である。各フレームは numpy 配列 (dtype は uint8) として得られるため、色反転は `255 - frame` という配列演算 1 つで式 2 を全画素・全チャンネルに適用できる。`cv2.bitwise_not()` は使用していない。縮小後のサイズは `(int(width * 0.3), int(height * 0.3))` で計算し、`cv2.resize` で縮小する。書き出しは `cv2.VideoWriter` を使い、コーデックには mp4 コンテナ用の mp4v を指定し、FPS は元動画の値をそのまま渡す。

3.6 課題 5: task5.spectrogram

`scipy.io.wavfile.read` で標準化周波数と信号を読み込み、ステレオの場合はチャンネル平均でモノラル化する。`scipy.signal.spectrogram` に窓関数 (`window`)、窓長 (`nperseg`)、オーバーラップ (`noverlap`) を渡して式 3 の $|X(m, k)|^2$ を計算し、式 4 で dB に変換する。表示は `matplotlib` の `pcolormesh` を使い、横軸を時間、縦軸を周波数、濃淡をパワーとするグラフとして保存する。窓長などのパラメータは引数として調節できるようにしてある。

3.7 課題 6: tokenize、build_2grams、task6a_2gram、task6b_2gram_no_stopwords

`tokenize` はテキストを小文字化し、正規表現 `[a-z]+` に一致する部分列を単語として返す。これにより句読点や数字が単語から分離される。`build_2grams` は単語リストの隣接 2 語を空白で連結した文字列のリストを返す。頻度集計は `collections.Counter` で行い、`most_common(10)` で上位 10 個を取り出して `matplotlib` の棒グラフとして保存する (`plot_top_2grams`)。6-b では、あらかじめ設計した stop words 集合 `STOP_WORDS` (冠詞・be 動詞・前置詞・接続詞・代名詞・助動詞など 90 語程度) に含まれない単語だけを残してから同じ処理を行う。

3.8 課題 7: task7.frame_differences ほか

`task7.frame_differences` は動画を先頭から読み、各フレームをグレースケール化して `np.int32` に変換する。`uint8` のまま減算すると負の値が桁あふれするため、符号付き整数への変換が必要である。前フレームとの差の絶対値の総和 `np.abs(gray - prev_gray).sum()` を変化量 (式 5) として配列に蓄積する。`absdiff()` は使用していない。検出関数は 3 つある。`detect_by_mean_std` は式 6 の閾値を計算し `np.where` で超過フレームを返す。`detect_by_top_peaks` は `np.argsort` の降順上位 n 個を返す。`detect_by_moving_average` は `np.convolve` で移動平均を計算し、その k' 倍を超えるフレームを返す。`save_boundary_frames` は検出フレームとその直前フレームを画像として保存し、目視確認に用いる。

3.9 課題 8: Video クラスとフレーム処理関数

`Video` クラスのコンストラクタは動画パスを受け取り、タイトル・ID・サムネイルを `None` で初期化する。`parse_filename` は課題 3 と同じ処理で `self.title` と `self.video_id` に代入する。`create_thumbnail` は課題 2 の代表フレーム選択を行い `self.thumbnail` に画像 (numpy 配列) を代入する。`get_info` は 4 つのインスタンス変数を辞書 `{"path", "title", "video_id", "thumbnail"}` として返す。

動画処理は `process_video(frame_func, out_path, scale)` に集約した。この関数は動画入出力 (読み込み・リサイズ・書き出し) だけを担い、フレーム 1 枚の変換は引数 `frame_func` に委ねる。フレーム変換関数として `invert_frame` (式 2 の色反転)、`grayscale_frame` (グレースケール化後、動画書き出しのため 3 チャンネルに戻す)、`binarize_frame` (グレースケール化後、閾値 128 で 2 値化し 3 チャンネルに戻す)

の 3 つを用意した。`create_inverse_video` は `process_video` に `invert_frame` を渡すだけで課題 4 と同じ処理を実現している。

4 実行結果

4.1 実行環境

プログラムは以下の環境で実行した。実行出力の全文は付録 B に無加工で示す。

表 1: 実行環境	
項目	内容
OS	macOS (Darwin 25.5.0)
Python	3.13.2
OpenCV (cv2)	4.12.0
numpy	2.4.4
matplotlib	3.10.6
scipy	1.15.2

データセットのパスは、課題指定の Colab パスを既定値とし、手元環境では環境変数 `DATASET_ROOT` で差し替えて実行した (プログラムの変更はない)。

4.2 課題 1: 動画の読み込み

`People - 84973.mp4` について、横 1280、縦 720、FPS 25.0、総フレーム数 241 が出力された。1280×720 は HD 解像度であり、再生時間は $241/25.0 = 9.64$ 秒に相当する。

4.3 課題 2: サムネイル画像の作成

代表フレームとしてフレーム 185 が選択され、図 1 のサムネイルが保存された。人物が画面中央付近に大きく写っており、動画の内容を代表する画像が得られている。



図 1: 選択されたサムネイル画像 (フレーム 185)

4.4 課題 3: ファイル名のパース

「People - 84973.mp4」に対して Title: People、ID: 84973 が、「Cat - 66004.mp4」に対して Title: Cat、ID: 66004 が出力され、いずれも課題の出力結果例と一致した。

4.5 課題 4: 色反転と動画の保存

Cat - 66004.mp4(1280×720、294 フレーム) から、色反転した 384×216 の動画 6324034_cat_reverse.mp4 が保存された。出力サイズは $1280 \times 0.3 = 384$ 、 $720 \times 0.3 = 216$ であり、指定どおり縦・横それぞれ 0.3 倍になっている。

4.6 課題 5: 音声の可視化

report_audio.wav(標本化周波数 44100 Hz、264600 サンプル、約 6.0 秒) のスペクトログラムを図 2 に示す。窓長 1024、オーバーラップ 768、ハン窓、dB 表示の設定で、時間-周波数領域に隠されたメッセージ「Good work」が判読できた。

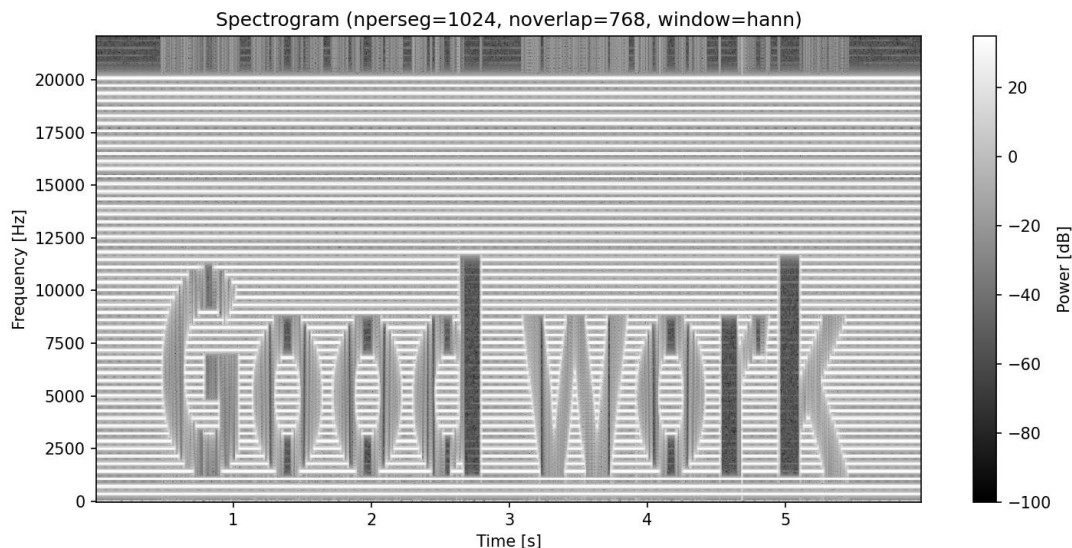


図 2: report_audio.wav のスペクトログラム (窓長 1024、オーバーラップ 768、ハン窓)

4.7 課題 6: n-gram

stop words を除去しない場合 (6-a) の上位 10 個の 2-gram を図 3 に、stop words を除去した場合 (6-b) を図 4 に示す。テキストの総単語数は 984、stop words 除去後は 614 であった。除去前の 1 位は「natural language」(15 回) で、「in the」「of the」などの機能語の組が上位に混在した。除去後は「natural language」「language processing」「machine translation」など内容語の組のみが上位を占めた。

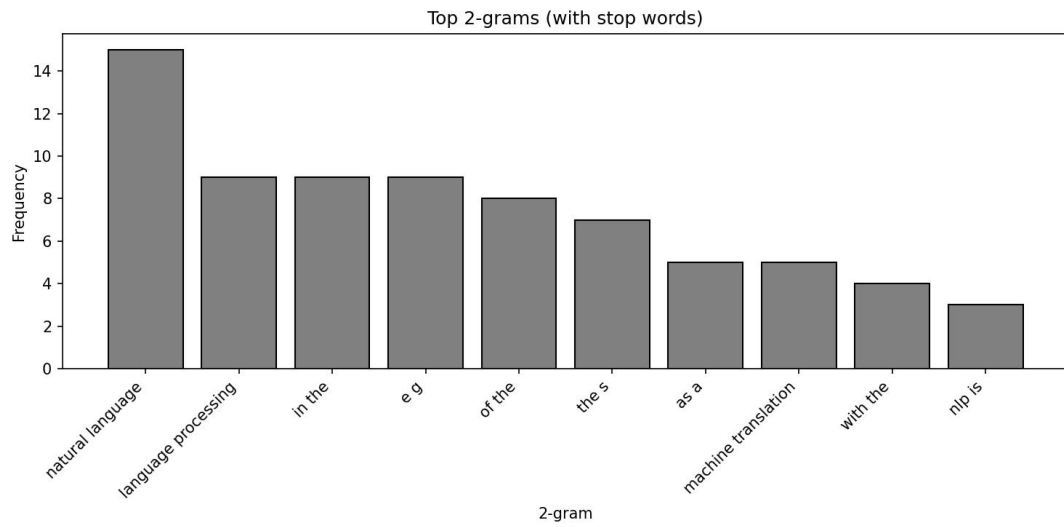


図 3: 2-gram 出現頻度上位 10 個 (stop words 除去なし)

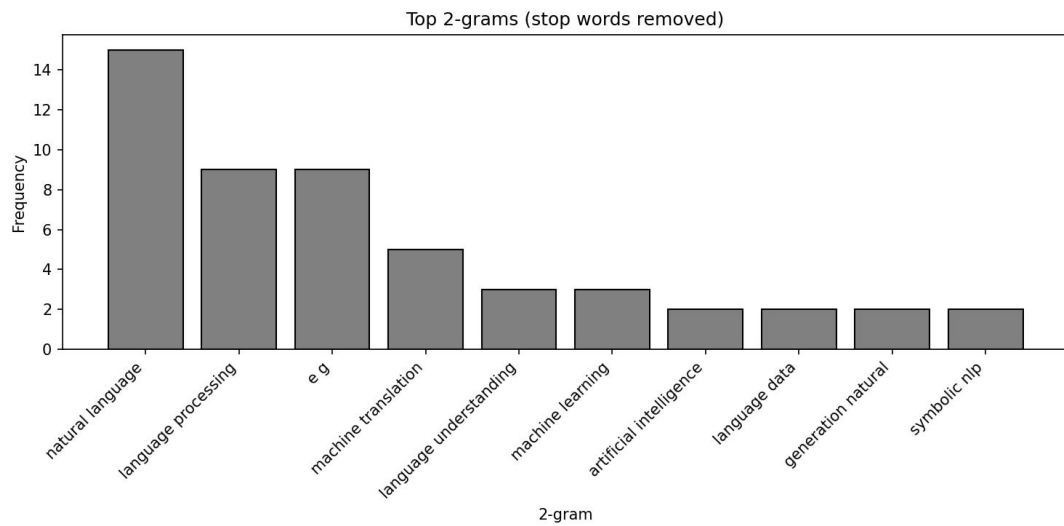


図 4: 2-gram 出現頻度上位 10 個 (stop words 除去あり)

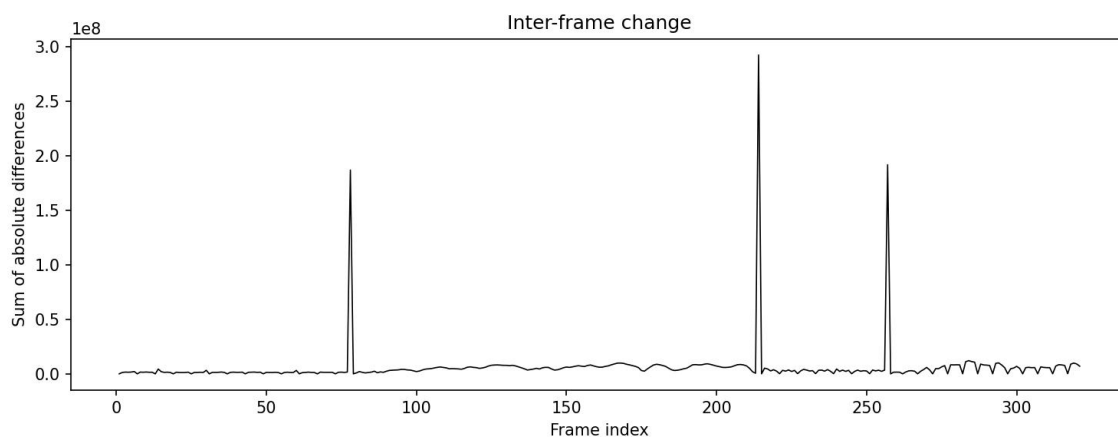


図 5: 隣接フレーム間の画素値変化量の推移

4.8 課題 7: 動画の変わり目検出

Detection.mp4 は 322 フレームからなり、隣接フレーム間の変化量は 321 個計算された。変化量の推移を図 5 に示す。3 本の突出したピークが確認できる。

3 方式の検出結果を表 2 に示す。いずれの方式でもフレーム 78、214、257 が変わり目として検出され、結果は一致した。

表 2: 閾値決定方式ごとの変わり目検出結果

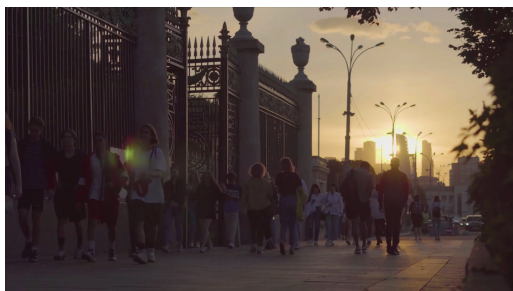
方式	閾値・条件	検出フレーム
平均 $+3\sigma$	$\theta = 71682248.1$	78, 214, 257
上位 3 ピーク	変化量降順 3 個	78, 214, 257
移動平均基準	窓 15、係数 4.0	78, 214, 257

検出されたフレームとその直前フレームの画像を図 6 に示す。いずれの箇所でも直前フレームと検出フレームで場面が完全に入れ替わっており、検出された 3 フレームが実際に動画の変わり目であることが確認できた。

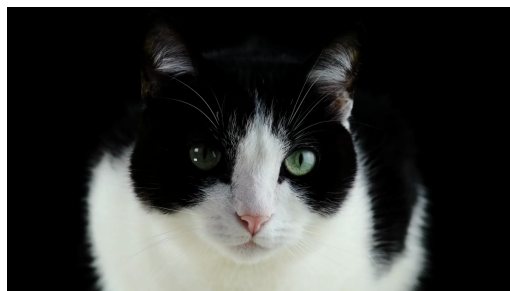
4.9 課題 8: Video クラスの動作確認

People-84973.mp4 で Video クラスをインスタンス化し、`parse_filename`、`create_thumbnail`、`get_info` が正しく動作すること (title: People、video_id: 84973、thumbnail: フレーム 185、形状 (720, 1280, 3)) を確認した。また、高階関数方式の `process_video` に二値化・グレースケール化・色反転の 3 関数をそれぞれ渡し、3 本の処理済み動画が生成されることを確認した。このうち色反転を渡した結果は課題 4 の出力と同一の処理である。

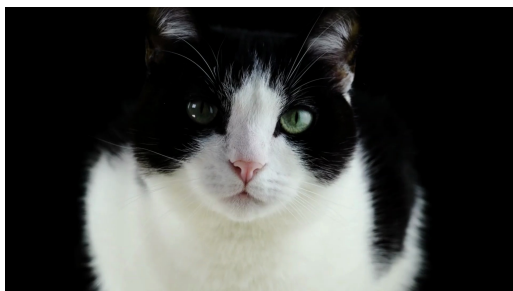
以上より、作成したプログラムは 8 つの課題すべてを満たして正しく動作している。詳細な検討は次節の考察で行う。



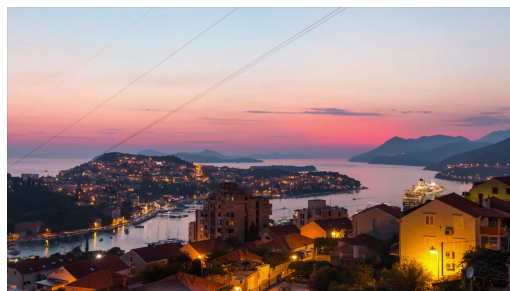
フレーム 77



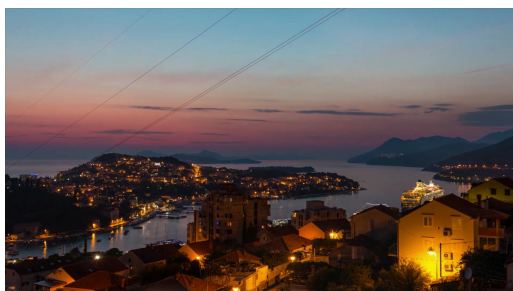
フレーム 78



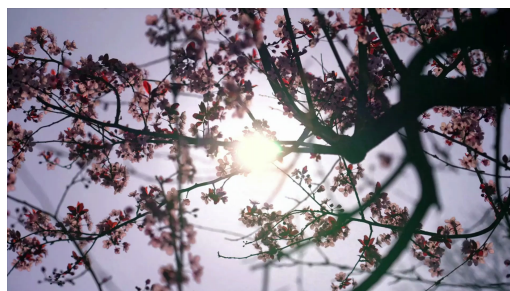
フレーム 213



フレーム 214



フレーム 256



フレーム 257

図 6: 検出された変わり目 (右列) とその直前フレーム (左列)

5 考察

5.1 サムネイル選択手法について

5.1.1 採用した代表フレーム選択基準の特徴

課題 2 では、中央フレームを選ぶ方法ではなく、動画全体の平均ヒストグラムに最も近いフレームを選ぶ方法を採用した。この基準の利点は、選択結果が「動画のどの位置か」ではなく「動画の内容」に基づくことである。中央フレーム法は実装が単純だが、中央がたまたま場面転換・手ぶれ・フェードの途中であった場合にも、その品質を評価せずに採用してしまう。一方、平均ヒストグラム法では、例外的な明るさ分布を持つフレームは平均から遠ざかるため選ばれにくく、動画の典型的な画面構成を持つフレームが選ばれる。

実際、People - 84973.mp4(総フレーム数 241) では中央フレームは 120 であるが、採用手法はフレーム 185 を選択した。すなわち両手法の選択は一致せず、採用手法は動画後半の人物が大きく写る場面を「より平均的な画面」と判定した。

5.1.2 採用手法の限界

平均ヒストグラム法はグレースケールの明るさ分布のみを比較するため、構図や被写体の意味内容は考慮しない。明るさ分布が平均的でも構図として無意味なフレームが選ばれる可能性は残る。また、複数の場面からなる動画では「全場面の平均」がどの場面とも異なる分布になり、代表性が損なわれることが考えられる。この点は、場面ごとに代表フレームを選ぶ(課題 7 の変わり目検出と組み合わせる)ことで改善できると考えられる。

5.2 スペクトログラムのパラメータについて

5.2.1 メッセージが読みやすかった設定

窓長 (nperseg)1024、オーバーラップ (noverlap)768(窓長の 75%)、ハン窓、dB 表示の組み合わせで、メッセージ「Good work」が最も明瞭に判読できた。標本化周波数 44100 Hz、窓長 1024 のとき、周波数分解能は $44100/1024 \approx 43$ Hz、時間分解能(ホップ幅 256 サンプル)は約 5.8 ms であり、約 6 秒の音声に対して横方向約 1000 列の十分な密度が得られる。

5.2.2 パラメータが不適切な場合との違い

窓長を極端に小さくすると周波数分解能が粗くなり、文字の縦方向の形状がつぶれる。逆に窓長を極端に大きくすると時間分解能が粗くなり、文字の横方向の形状がつぶれる。これは式 3 の窓長 N に関する時間分解能と周波数分解能のトレードオフそのものである。また、dB 変換(式 4)を行わず線形スケールで表示すると、パワーの大きい少数の成分だけが濃く表示され、メッセージ部分とのコントラストが失われて判読できなくなる。

5.3 n-gram と stop words について

5.3.1 stop words 除去の効果

図 3 と図 4 の比較から、除去前は上位 10 個のうち「in the」「of the」「as a」「with the」など機能語を含む組が半数を占めるのに対し、除去後は「natural language」「language processing」「machine translation」「artificial intelligence」など、テキストの主題(自然言語処理)を直接表す内容語の組だけが残った。この

ことから、stop words 除去は 2-gram 分析において文章の内容的特徴を浮かび上がらせる効果があるといえる。

5.3.2 トークン化の設計と「e g」の出現

除去後の上位に「e g」という 2-gram が現れた (9 回)。これは原文中の略語「e.g.」がトークン化 (正規表現 $[a-z]^+$) によって「e」と「g」に分割されたものである。1 文字語「e」「g」は設計した stop words 集合に含めなかったため、両者が隣接する 2-gram として集計された。内容分析の目的からは「e.g.」は機能語に近く、stop words に 1 文字語や略語を追加するか、トークン化の段階で「e.g.」を 1 語として扱う設計が望ましいと考えられる。この点は stop words の設計がトークン化の仕様と不可分であることを示している。

5.3.3 2-gram 構成方式の影響

本実装では stop words を除去した後の単語列から 2-gram を構成した。このため、原文では stop words を挟んで離れていた 2 語 (例: 「generation of natural」の「generation」と「natural」) が新たな隣接ペア「generation natural」として集計される。実際、図 4 には原文に連続表現として存在しない「generation natural」が出現している。共起関係を広く拾う目的にはこの方式が適するが、原文に実在する接続表現のみを数えたい場合は、2-gram を構成してから stop words を含む組を捨てる方式を採るべきである。両方式は目的に応じて使い分けるべき設計選択である。

5.4 変わり目検出の閾値決定法について

5.4.1 3 方式の比較

表 2 のとおり、平均 $+3\sigma$ 方式・上位 3 ピーク方式・移動平均基準方式の検出結果は完全に一致した。これは図 5 に見られるように、変わり目の変化量 (最大で約 2.3×10^8) がショット内の変化量 (概ね 10^7 以下) より 1 桁以上大きく、どの合理的な閾値でも分離できるためである。

ただし 3 方式の性質は異なる。上位 n ピーク方式は変わり目の個数が既知でなければ使えず、汎用性に欠ける。平均 $+k\sigma$ 方式は個数の事前知識を要しないが、動画全体で 1 つの閾値を使うため、動きの激しいショットと静かなショットが混在すると誤検出・見逃しが起きやすい。移動平均基準方式は閾値が局所的な変化量に追従するため、この問題に最も頑健である。本課題の動画では差が現れなかったが、ショット内の動きが大きい実映像では方式間の差が現れると考えられる。

5.4.2 検出結果の妥当性

図 6 の目視確認により、検出された 3 フレームすべてで場面が完全に入れ替わっていることを確認した。誤検出 (変わり目でないフレームの検出) も見逃し (4 区間構成に必要な 3 境界のうち検出されなかったもの) もなく、本課題の動画に対しては単純なフレーム間差分の総和で十分な精度が得られた。

5.5 関数ベース設計とクラスベース設計の比較

5.5.1 データと処理の凝集

課題 1~7 のような関数ベースの設計では、動画のパス・タイトル・ID・サムネイルといった関連するデータを関数間で個別に受け渡す必要があり、扱う動画が増えると変数管理が煩雑になる。課題 8 の Video

クラスでは、これらが 1 つのインスタンスに凝集され、「その動画に対する操作」がメソッドとして明示されるため、複数の動画を扱うプログラムでも見通しがよい。get_info() のように状態を辞書としてまとめて返す窓口を用意することで、クラス内部の変数名の変更が利用側に波及しにくくなる利点もある。

5.5.2 高階関数による処理の分離

色反転動画の生成を「フレーム変換」と「動画入出力」に分離したことで、二値化・グレースケール化の追加はフレーム変換関数を 1 つ書くだけで済んだ。仮に分離しない設計であれば、動画の読み込み・リサイズ・書き出しという同一のコードを処理の種類だけ複製することになり、コーデック変更などの修正が全コピーに波及する。処理の差し替え可能性という点で、高階関数方式はクラス設計と補完的に働いている。一方、フレーム変換関数側は「1 フレームを受け取り 1 フレームを返す」という暗黙の契約に従う必要があり、契約を型として明示できない Python では、関数の仕様をドキュメントで共有することが重要になる。

5.6 サムネイル選択手法の定量比較

これまでの考察では、採用した平均ヒストグラム法と中央フレーム法の違いを定性的に述べた。本節ではこれを定量比較に拡張し、代表フレーム選択の 4 手法を同一動画に適用して、選択結果・評価値の推移・指標間の相互評価・計算時間を比較する。実験には追加スクリプト exp01_thumbnail.py(付録参照)を用いた。対象は課題 2 と同じ People - 84973.mp4(1280 × 720 画素、25 fps、241 フレーム)であり、実行環境は OpenCV 4.12.0、NumPy 2.4.4 である。

5.6.1 比較手法と評価指標の定義

フレーム番号を i ($i = 0, 1, \dots, 240$)、総フレーム数を $N = 241$ と書く。比較した手法は以下の 4 つである。

第一の中央フレーム法は、フレームの内容を評価せずに $\lfloor N/2 \rfloor = 120$ 番のフレームを選ぶ。第二の平均ヒストグラム法は課題 2 で採用した手法であり、task2_thumbnail_frame と同一のロジックで、sample_step = 5 で間引いたフレーム集合 $\mathcal{S} = \{0, 5, \dots, 240\}$ (49 フレーム) の各フレームについて 64 ビンのグレースケール正規化ヒストグラム h_i を計算し、式 7 の評価値 d_i が最小のフレームを選ぶ。

$$d_i = \|h_i - \bar{h}\|_2, \quad \bar{h} = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} h_j \quad (7)$$

ここで h_i は 64 次元のヒストグラムベクトル、 $\bar{h}$ はその平均である。第三の鮮明度法は、ぼけの少ないフレームほど高周波成分が多く残ることを利用し、グレースケール画像 I_i にラプラシアンフィルタを適用した結果の分散 (式 8) が最大のフレームを選ぶ。

$$S_i = \text{Var} [\nabla^2 I_i] \quad (8)$$

$\nabla^2 I_i$ の計算には cv2.Laplacian を用いた。第四のカラフルネス法は、Hasler-Suesstrunk のカラフルネス指標 (式 9) が最大のフレームを選ぶ。

$$M_i = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3 \sqrt{\mu_{rg}^2 + \mu_{yb}^2}, \quad rg = R - G, \quad yb = \frac{R + G}{2} - B \quad (9)$$

ここで R, G, B は各画素の色成分、 μ と σ はそれぞれ rg, yb の画像内での平均と標準偏差である。 M_i は彩度の豊かさを 1 つの実数に要約する指標であり、値が大きいほど色鮮やかである。鮮明度法とカラフルネス法は全 241 フレームを評価対象とした。

5.6.2 選択フレームと評価値の推移

表 3 に各手法の選択フレームと計算時間を、図 7 に各評価値の全フレームにわたる推移と選択位置を、図 8 に選択されたフレーム画像を示す。

表 3: 各手法の選択フレームと計算時間 (People - 84973.mp4、全 241 フレーム)

手法	選択フレーム	評価フレーム数	総計算時間 [s]	1 フレームあたり [ms]
中央フレーム法	120	—	1×10^{-6}	—
平均ヒストグラム法 (採用)	185	49	4.84	98.8
鮮明度法	208	241	6.06	25.1
カラフルネス法	8	241	11.73	48.7

4 手法の選択フレームは 8、120、185、208 とすべて異なった。同一の動画に対しても、選択基準を変えるだけでサムネイルが完全に入れ替わることが定量的に確認できた。

図 7 (b) より、採用手法の評価値 d_i は動画冒頭 (最大値 0.0207 はフレーム 10) と末尾で大きく、フレーム 170~190 付近で小さい。最小値 0.00706 はフレーム 185 で得られ、最小値の 5% 以内に入るフレームは 185 と 186 の 2 枚のみであり、選択は明確に定まっている。(c) のラプラシアン分散 S_i は大部分のフレームで 215~230 の範囲にあり、最小値 194.3 は最終フレーム 240 で生じた。最大値 230.1 を与えるフレーム 208 が鮮明度法の選択である。(d) のカラフルネス指標 M_i は動画中盤 (最小値 27.48 はフレーム 78) で低く、冒頭と後半で高い。最大値 28.24 を与えるフレーム 8 がカラフルネス法の選択である。

図 8 の 4 枚はいずれも同一ショット内の画像であり構図は類似するが、人物の配置と逆光の強さが異なる。フレーム 185 は課題 2 で出力したサムネイルそのものである。

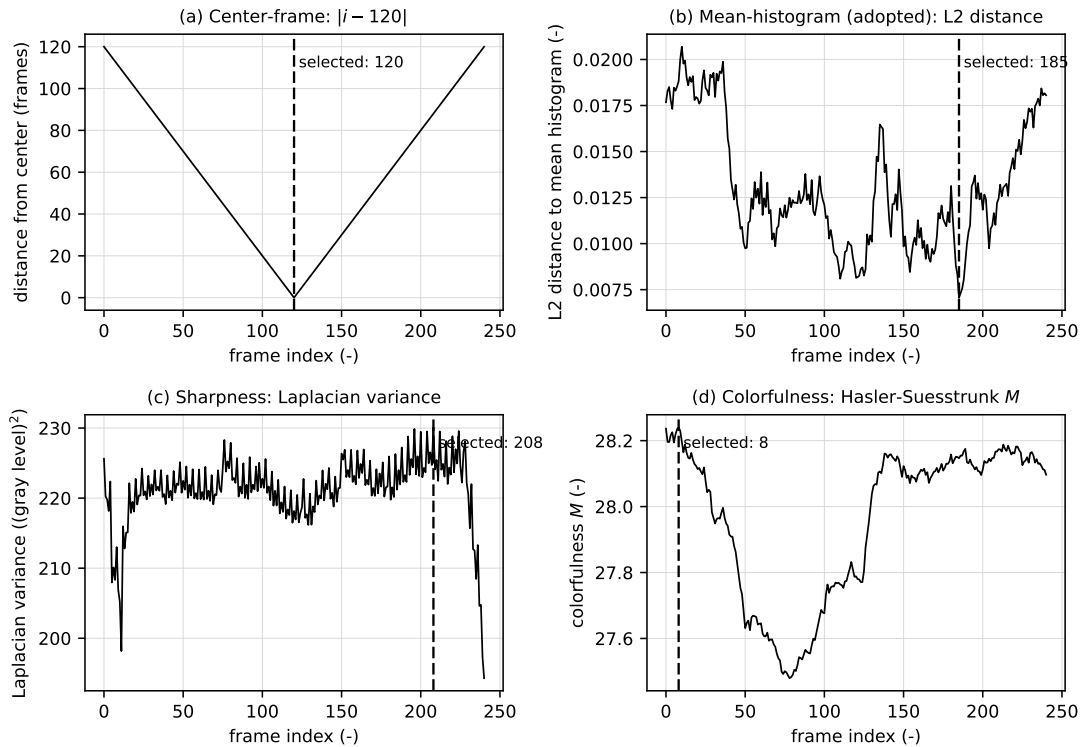


図 7: 4 手法の評価値の全フレーム推移。(a) 中央フレーム法 (中央からの距離 $|i - 120|$)、(b) 平均ヒストグラム法 (平均ヒストグラムへの L2 距離 d_i)、(c) 鮮明度法 (ラプラシアン分散 S_i)、(d) カラフルネス法 (Hasler-Suesstrunk 指標 M_i)。縦の破線は各手法の選択フレーム位置を示す。



図 8: 各手法が選択したフレーム画像。(a) 中央フレーム法 (フレーム 120)、(b) 平均ヒストグラム法 (フレーム 185)、(c) 鮮明度法 (フレーム 208)、(d) カラフルネス法 (フレーム 8)。

5.6.3 指標間の相互評価

各手法の選択フレームを、他の手法の評価指標でも採点した結果を表 4 に示す。表中の百分位は、全 241 フレーム中で当該値以下のフレームが占める割合であり、 d_i は小さいほど、 S_i と M_i は大きいほど良い値である。

表 4: 各手法の選択フレームにおける 3 指標の値と百分位。百分位は全 241 フレーム中で当該値以下のフレームが占める割合 [%] を表す。

手法	フレーム	d_i	百分位	S_i	百分位	M_i	百分位
中央フレーム法	120	0.00814	2.5	218.2	14.9	27.79	30.7
平均ヒストグラム法 (採用)	185	0.00706	0.4	221.2	44.8	28.15	78.0
鮮明度法	208	0.01101	31.5	230.1	100.0	28.17	89.2
カラフルネス法	8	0.01882	92.9	213.0	6.2	28.24	100.0

採用手法が選んだフレーム 185 は、 d_i が全フレーム最小 (百分位 0.4%) であることに加えて、鮮明度 S_i の百分位 44.8% (ほぼ中央値)、カラフルネス M_i の百分位 78.0% と、最適化の対象としていない指標でも中位以上の値を示した。すなわち、明るさ分布の代表性を基準に選んだフレームが、鮮明度・彩度の面でも大きな欠点を持たないことが確認できた。

中央フレーム 120 の d_i は 0.00814 で、最小値より 15.3% 大きいだけである (百分位 2.5%)。この動画に限れば中央フレームも明るさ分布の代表性は高いが、これは本動画が場面転換のない単一ショットで、画面構成が緩やかにしか変化しないためと考えられる。一方で鮮明度は百分位 14.9% と下位にあり、中央フレーム法がフレームの品質を評価しないという欠点は数値にも表れている。

カラフルネス法が選んだフレーム 8 は、 M_i こそ最大 (28.24) だが、 d_i の百分位が 92.9% と明るさ分布が最も非典型的な部類に属し (d_i 最大のフレーム 10 の直近でもある)、鮮明度も百分位 6.2% と最もぼけた部類にある。単一指標の最大化は、その指標以外の品質を大きく犠牲にし得ることを示す実例である。鮮

明度法のフレーム 208 は S_i 最大 (230.1) かつ M_i 百分位 89.2% と良好だが、 d_i は百分位 31.5% にとどまり、明るさ分布の代表性では採用手法に劣る。

指標としての判別力にも大きな差があった。変動係数 (標準偏差を平均で割った値) は d_i が 25.7% であるのに対し、 S_i は 2.26%、 M_i は 0.83% にすぎない。特に M_i は全変動幅 (最大値と最小値の差) が平均値の 2.7% しかなく、わずかな差で選択が決まるため、この動画に対する分離能は乏しい。採用手法の評価値 d_i は相対変動が最も大きく、フレーム間の差を最も明瞭に区別できる指標であった。

また、3 指標の全フレームにわたる相関は弱く、Pearson 相関係数は d_i - S_i 間で -0.384 、 d_i - M_i 間で $+0.332$ 、 S_i - M_i 間で -0.086 、Spearman 順位相関係数はそれぞれ -0.161 、 $+0.314$ 、 $+0.053$ であった。図 9 に min-max 正規化した 4 つの評価値を重ねて示すが、極小・極大の位置は指標ごとに異なる。3 指標は互いにほぼ独立した性質 (代表性・鮮明度・彩度) を測っており、単一指標で他を代替することはできない。したがって、サムネイルの総合品質をさらに高めるには、複数指標の重み付き結合が改善方向となる。

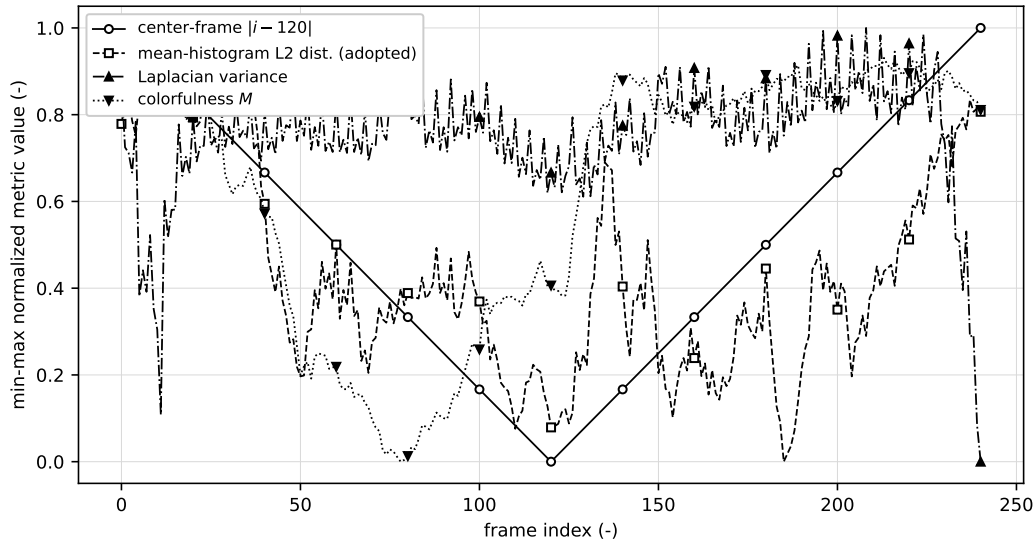


図 9: min-max 正規化した 4 手法の評価値の重ね描き。中央からの距離 $|i - 120|$ と d_i は小さいほど、 S_i と M_i は大きいほど良い値である。

5.6.4 サンプリング間隔と計算時間

採用手法の `sample_step = 5` の妥当性も検証した。全フレームを評価に用いた場合 (`sample_step = 1`) の選択もフレーム 185 で一致し、間引きによる選択の変化は 0 フレームであった。すなわち、この動画では評価フレーム数を 241 枚から 49 枚へ 1/5 に減らしても選択結果は変わらず、間引きは計算量削減の手段として妥当である。

計算時間は表 3 のとおりである。中央フレーム法は添字計算のみで約 10^{-6} s と事実上ゼロである。内容に基づく 3 手法では、採用手法が 4.84 s と最も短く、鮮明度法 (6.06 s) の 79.9%、カラフルネス法 (11.73 s) の 41.3% であった。ただし 1 フレームあたりの指標計算は採用手法が 98.8 ms と最も重く (鮮明度法 25.1 ms の 3.9 倍、カラフルネス法 48.7 ms の 2.0 倍)、総時間が最短になるのは間引きの効果である。ヒストグラム計算に汎用の `numpy.histogram` を用いていることが 1 フレームあたりの時間を押し上げており、OpenCV の専用関数 `cv2.calcHist` に置き換えることで短縮できると考えられる。なお、動画のデコードには全手法共通で 4.64 s (241 フレーム、19.3 ms/フレーム) を要しており、内容に基づく選択手法の実行時間はデコード時間と同程度以上になる点も設計上考慮すべきである。

5.6.5 キーフレーム抽出研究における位置付け

動画要約の近年のサーベイ [1] では、キーフレームは動画を代表する最も重要なフレームと定義され、静的要約 (サムネイルを含む) の生成では色などの低水準特徴の変化に基づく抽出が基本的な構成要素とされている。採用した平均ヒストグラム法は、この分類における低水準特徴 (明るさ分布) の代表性基準に相当する。また、深層学習に基づく動画要約のサーベイ [2] が扱うように、近年は代表性や多様性を学習によって最適化する手法が発展しているが、これらは学習データと計算資源を要する。本課題のように単一動画から 1 枚のサムネイルを選ぶ用途では、本実験で示したとおり、低水準特徴による選択でも鮮明度・彩度の百分位がそれぞれ 44.8%・78.0% と均衡の取れたフレームが得られる。

以上より、採用手法は (1) 評価値の変動係数が 25.7% と最も大きく判別力に優れること、(2) 最適化の対象外の指標でも中位以上の値を示し極端な欠点がないこと、(3) 内容に基づく 3 手法の中で計算時間が最短であることの 3 点から、比較した手法の中で最も妥当なサムネイル選択基準であると結論できる。

5.7 スペクトログラムのパラメータ感度の定量評価

5.2 節では、課題 5 のスペクトログラムのパラメータ (窓長・窓関数・表示スケール) がメッセージの判読性に与える影響を定性的に述べた。本節では、追加実験 `exp02_spectrogram.py` により、この影響を画像コントラスト指標で定量化した結果を考察する。実験ソースコードと実行出力の全文は付録に無加工で示す。

5.7.1 実験方法とコントラスト指標の定義

`report_audio.wav` (標本化周波数 $f_s = 44100$ Hz、264600 サンプル、6.00 秒) に対して、`scipy.signal.spectrogram`[3] を用いて以下の 3 系列の設定でスペクトログラムを計算した。

- 窓長 `nperseg` $\in \{128, 512, 1024, 4096, 16384\}$ (ハン窓、オーバーラップは窓長の 75% に固定)
- 窓関数 $\in \{\text{hann}, \text{boxcar}, \text{blackman}\}$ (`nperseg = 1024` に固定)
- 表示スケール $\in \{\text{dB}, \text{線形}\}$ (`nperseg = 1024`、ハン窓に固定)

判読性の定量指標には、実際に紙面に表示される濃淡そのものを対象とするため、表示画素値を用いた。dB 表示ではパワーの dB 値 (式 4) を、最大値 $P_{\max}$ からダイナミックレンジ $D = 80$ dB の範囲で $[0, 1]$ に正規化した値

$$I(m, k) = \min \left(1, \max \left(0, \frac{P_{\text{dB}}(m, k) - (P_{\max} - D)}{D} \right) \right) \quad (10)$$

を表示画素値と定義する (m は時間格子、 k は周波数ビン)。これは図の `pcolormesh` に与える表示範囲 (`vmin =  $P_{\max} - D$` 、`vmax =  $P_{\max}$`) と同一の規則であり、図の見た目と指標が対応する。線形表示では $I(m, k) = S(m, k) / S_{\max}$ (S はパワー、 $S_{\max}$ はその最大値) とした。

メッセージが目視で確認できる帯域 1–12 kHz の画素集合を B とし、2 つのコントラスト指標を定義した。第 1 は RMS コントラスト

$$C_{\text{RMS}} = \sqrt{\frac{1}{|B|} \sum_{(m, k) \in B} (I(m, k) - \bar{I})^2} \quad (11)$$

($\bar{I}$ は帯域内平均) であり、表示濃淡の広がりを直接測る。第 2 は外れ値に頑健な 5/95 パーセンタイル版のミケルソンコントラスト

$$C_{\text{M}} = \frac{I_{95} - I_5}{I_{95} + I_5} \quad (12)$$

(I_{95} 、 I_5 は帯域内画素値の 95、5 パーセンタイル) である。全設定の測定結果を表 5 に示す。表中の $\Delta f = f_s/N$ は周波数ビン間隔、 $\Delta t = H/f_s$ ($H = N/4$ はホップ幅) は時間格子間隔である。

表 5: STFT パラメータごとのコントラスト指標 (帯域 1–12 kHz。基準設定 hann、nperseg = 1024、dB は各区分に重複して現れる)

区分	窓関数	スケール	nperseg	Δf [Hz]	Δt [ms]	C_{RMS}	C_{M}
窓長	hann	dB	128	344.53	0.726	0.2995	0.9292
窓長	hann	dB	512	86.13	2.902	0.2966	1.0000
窓長	hann	dB	1024	43.07	5.805	0.2719	1.0000
窓長	hann	dB	4096	10.77	23.220	0.2422	1.0000
窓長	hann	dB	16384	2.69	92.880	0.1961	1.0000
窓関数	hann	dB	1024	43.07	5.805	0.2719	1.0000
窓関数	boxcar	dB	1024	43.07	5.805	0.1617	0.4101
窓関数	blackman	dB	1024	43.07	5.805	0.3063	1.0000
スケール	hann	dB	1024	43.07	5.805	0.2719	1.0000
スケール	hann	線形	1024	43.07	5.805	0.2230	1.0000

なお、図 10 から分かるように、本音声のメッセージは「明るい背景 (広帯域雑音) から文字形状にエネルギーを欠落させた」ノッチ型の埋め込みであり、文字は白地に黒く現れる。以下の考察はこの埋め込み形態を前提とする。

5.7.2 窓長の感度と不確定性関係

窓長を変えた 5 設定のスペクトログラムを図 10 に、コントラスト指標の推移を図 11 に示す。 C_{RMS} は nperseg = 128 の 0.2995 から 16384 の 0.1961 まで単調に減少し、最小設定と最大設定の間で約 35% 低下した。図 10 の目視でも、4096 で文字の時間方向のにじみが現れ、16384 では文字がほぼ塗りつぶされて判読困難となっており、指標の低下と一致する。

この挙動は時間分解能と周波数分解能のトレードオフで説明できる。窓長 N の窓の時間幅は $T = N/f_s$ 、周波数ビン間隔は $\Delta f = f_s/N$ であり、積は $T \cdot \Delta f = 1$ で一定である。すなわち一方を細かくすれば必ず他方が粗くなる。これは連続時間信号の不確定性関係 $\sigma_t \sigma_f \geq 1/(4\pi)$ (ガボール限界) の離散版に相当し、式 3 の窓長 N をどう選んでも両分解能を同時には改善できない。実測値でも、表 5 の $\Delta f \cdot \Delta t$ は全窓長で 0.25 (例: nperseg = 128 では $344.53 \text{ Hz} \times 0.000726 \text{ s} = 0.250$) と一定であり、格子の細かさの積が保存されていることが確認できる。

判読性が窓長に対して非対称に振る舞う点が本実験の重要な知見である。図 10 の文字の縦ストロークの時間幅は約 0.1 秒であるのに対し、nperseg = 16384 の窓時間幅は $T = 16384/44100 = 371.5 \text{ ms}$ とストローク幅の 3 倍を超えるため、ノッチが窓内で背景雑音と平均化されて塗りつぶされる。nperseg = 4096 ($T = 92.9 \text{ ms}$) がストローク幅と同程度となる劣化の始まりであり、実際に C_{RMS} は基準設定 1024 の 0.2719 から 0.2422 へ低下している。一方、周波数方向は文字の高さが約 11 kHz、輪郭の構造が 1 kHz 程度と大きいため、nperseg = 128 ($\Delta f = 344.5 \text{ Hz}$ 、帯域内ビン数 32) でも判読性は保たれ、 C_{RMS} は最大の 0.2995 となった。すなわち本音声のメッセージは周波数分解能よりも時間分解能への要求が厳しく、窓長は小さめに外す方が安全である。ただし 128 と 512 の差は約 1% (0.2995 対 0.2966) と小さく、図の目視では 512–1024 の文字の輪郭が最も鮮明であることから、 C_{RMS} の微差のみで 128 を最良と断定することはできない。

C_{M} は 512 以上で 1.0000 に飽和した。これは文字ノッチの画素が表示下限 $P_{\text{max}} - 80 \text{ dB}$ に到達し、式 10 の下限クリップによって $I_5 = 0$ となるためであり、ノッチ深さが表示ダイナミックレンジを超えている

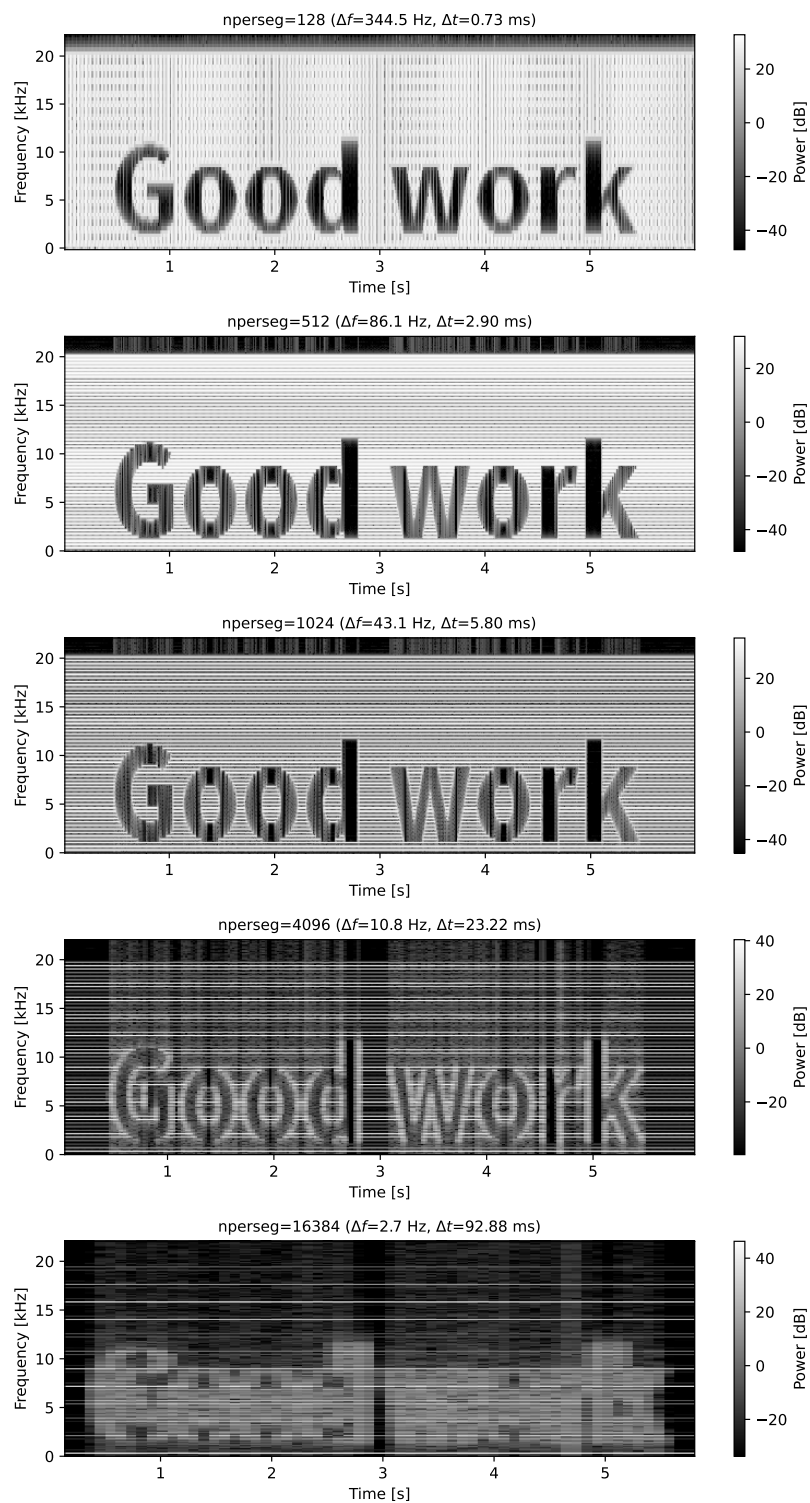


図 10: 窓長 $nperseg$ を変えたスペクトログラムの比較 (ハン窓、オーバーラップ 75%、dB 表示)

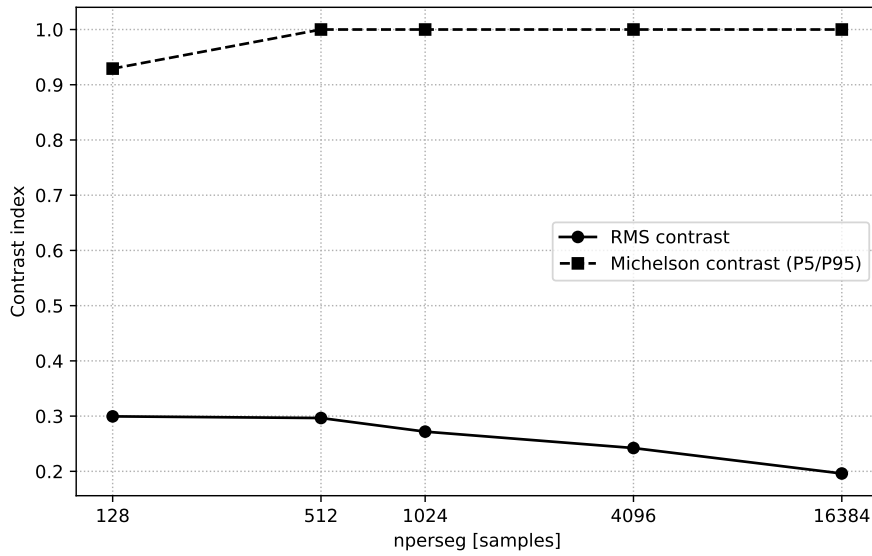


図 11: 窓長に対するコントラスト指標の推移 (実線: C_{RMS} 、破線: C_{M})

ことを意味する。128 でのみ 0.9292 に低下したのは、 $\Delta f = 344.5$ Hz の周波数方向の平滑化によりノッチの一部が表示下限まで下がりきらないためである。

5.7.3 窓関数の影響とスペクトルリーク

窓関数を変えた 3 設定の比較を図 12 に示す。 C_{RMS} は blackman 0.3063 > hann 0.2719 > boxcar 0.1617 の順となり、boxcar(矩形窓) は hann に対して約 41% 低い。図 12 でも boxcar の文字は黒ではなく灰色に浮いており、判読性が明確に劣る。

この差はスペクトルリーク (サイドローブ特性) で説明できる。矩形窓の第 1 サイドローブは約 -13 dB と高く、周囲の帯域の背景雑音のエネルギーが文字ノッチの周波数ビンへ漏れ込み、ノッチを埋めてしまう。 C_{M} が boxcar でのみ 0.4101 と大きく低下している (I_5 が表示下限から浮いている) ことは、ノッチの底がリークで持ち上げられたことを直接示す。一方 blackman のサイドローブは約 -58 dB と hann(約 -31 dB) よりさらに低く、 C_{RMS} は hann 比で約 12.7% 向上した。blackman は主ローブ幅が hann より広く周波数方向のぼけは増えるが、前項で述べたとおり本メッセージは周波数分解能への要求が緩いため、悪影響が現れない。エネルギーの欠落として埋め込まれたパターンの可視化では、主ローブ幅よりもサイドローブの低さが支配的であるといえる。

5.7.4 表示スケールの影響と指標の性質

dB 表示と線形表示の比較を図 13 に示す。 C_{RMS} は dB 表示 0.2719 に対し線形表示 0.2230 と約 18% の低下であった。5.2.2 項では線形表示によりメッセージ部分とのコントラストが失われると述べたが、定量実験の結果、本音声に限っては低下は限定的であり判読も可能であった。これは、メッセージがノッチ型 (文字部分のパワーがほぼ 0) で埋め込まれており、かつ帯域内の背景雑音のパワーが比較的一様であるため、線形スケールでも文字と背景の差が保たれるからである。成分間のダイナミックレンジが数十 dB に及ぶ一般の音声・音楽信号では、線形表示は最大成分以外の濃淡を潰すため、dB 変換の必要性は本実験の結果より大きくなると考えられる。

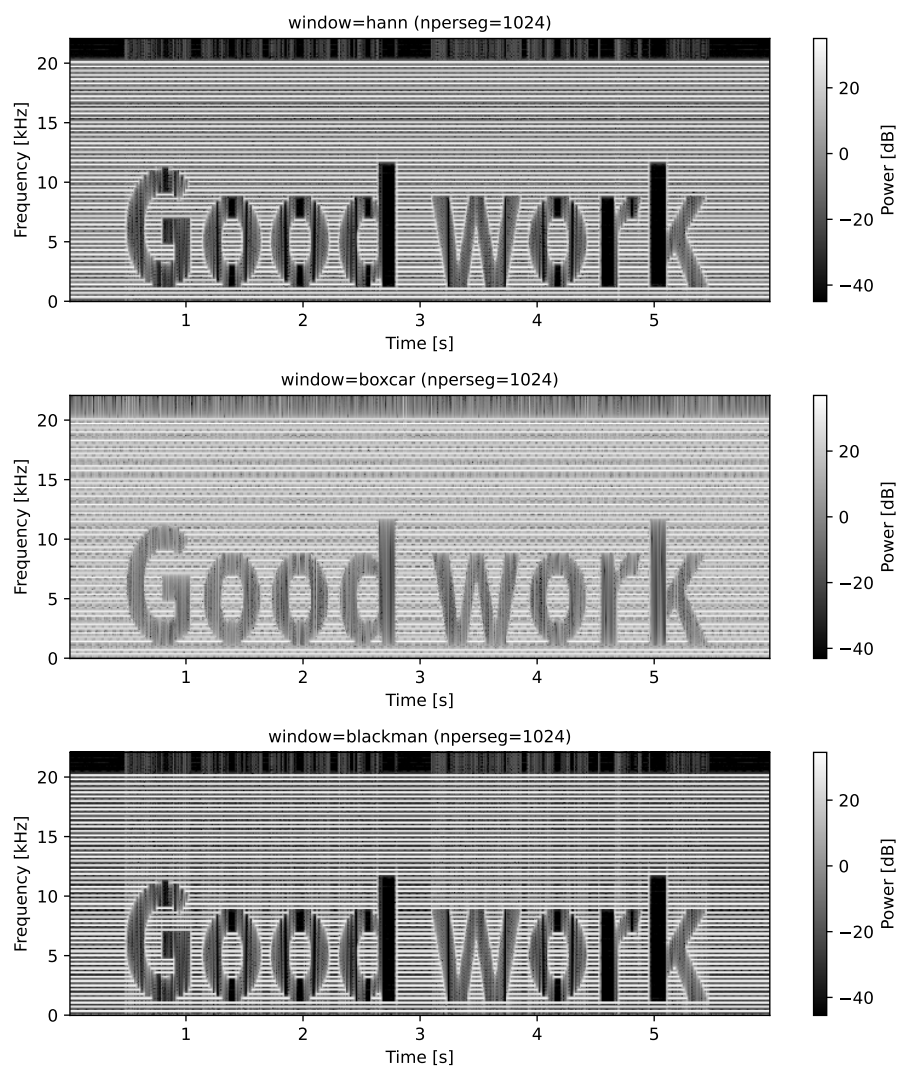


図 12: 窓関数によるスペクトログラムの比較 ($n_{\text{perseg}} = 1024$ 、dB 表示)

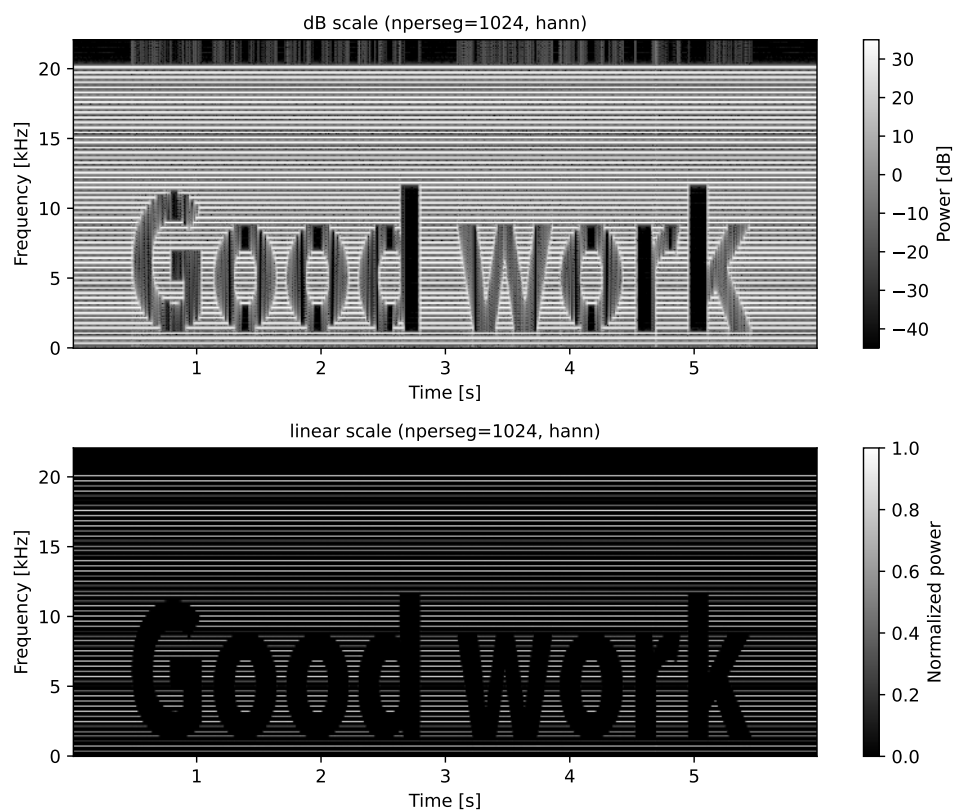


図 13: dB 表示 (上) と線形表示 (下) の比較 ($nperseg = 1024$ 、ハン窓)

指標の性質にも注意が必要である。表 5 のとおり C_M は線形表示でも 1.0000 となり、表示の暗さ・濃淡レンジの圧縮をまったく検出できていない。式 12 は分子と分母が画素値に対して同時にスケールするため、画素値全体が定数倍されても値が変わらない (スケール不変性)。可読性は紙面上の濃淡差の絶対量に依存するため、表示画素値の分散を直接測る C_{RMS} (式 11) の方が本目的の指標として適切である。 C_M はノッチ深さの飽和判定 (前々項) やリークによる底上げの検出 (前項) には有効であり、2 指標は相補的に用いるべきである。

5.7.5 最良設定と採用設定の妥当性

以上を総合すると、測定した範囲での最良設定は blackman 窓・nperseg = 512-1024・dB 表示 ($C_{RMS} = 0.297-0.306$) である。課題 5 で採用したハン窓・nperseg = 1024・オーバーラップ 75%・dB 表示は $C_{RMS} = 0.2719$ でこれに近く、boxcar(0.1617) や nperseg = 16384(0.1961) のような判読困難な領域から十分離れた妥当な設定であったことが定量的に裏付けられた。

また、本音声のようにスペクトログラム上へ視覚パターンを描く埋め込みは、音声ステガノグラフィの系統的レビュー [4] における変換領域 (周波数領域) 埋め込みの一種と位置づけられる。本実験の結果は、この種の埋め込みを受信側で復元するための条件を、窓の時間幅 T がパターンの時間方向の最小構造 (本音声では約 0.1 秒) を下回り、かつ窓関数のサイドローブがノッチ深さに対して十分低いこと、として数値的に与えるものである。

5.8 色反転処理の実装方式と実行時間

課題 4 の色反転は、numpy のベクトル演算 `255 - frame` によりフレーム全体を一括で計算する方式を採用した。この設計選択が実行時間の面でどの程度有利かを定量化するため、追加実験 (実験ソースは付録の `exp03_invert_timing.py`) として、(1) 実装方式 4 種の 1 フレームあたり実行時間の比較、(2) 画素数を変えたときのスケーリング特性の計測、(3) 動画全体の処理時間に占める反転処理の割合の計測を行った。計測対象は Cat - 66004.mp4(1280×720、294 フレーム、29.97 fps) である。時間計測には `timeit` を用い、計測前に 3 回の空実行でキャッシュと遅延初期化の影響を除いたうえで、1 セットの総時間が約 0.2 秒になるよう 1 セットあたりの実行回数 (number) を自動決定し、7 セット (`repeat=7`) の 1 回あたり時間の中央値を代表値とした。実行環境は表 1 と同一 (Apple Silicon、Python 3.13.2、numpy 2.4.4、OpenCV 4.12.0) である。

なお、OpenCV の `cv2.bitwise_not()` は本課題の制約により提出プログラムでは使用できない。本実験では提出プログラムには一切含めず、最適化された既製実装がどの程度の速度を出すかを知るための速度基準としてのみ参考計測した。

5.8.1 比較した実装方式と計測条件

色反転は、8 bit 画像の各画素値 p に対して

$$p' = 255 - p \quad (13)$$

を全画素・全チャンネルに適用する処理である。式 13 の計算内容は同一のまま、「どの層でループを回すか」だけが異なる次の 4 方式を比較した。

1. 方式 (a): `255 - frame`。numpy のブロードキャストにより配列全体を一括計算する。提出プログラムの採用手法であり、出力用の新しい配列が毎回確保される。

2. 方式 (b): `np.subtract(255, frame, out=buf)`。計算内容は方式 (a) と同じだが、事前に確保した出力バッファ `buf` を再利用することで、呼び出しごとのメモリ確保を省く。
3. 方式 (c): Python の 3 重ループ。for 文で行・列・チャンネルを走査し、画素成分ごとに式 13 を計算する。フルサイズでは時間がかかりすぎるため 160×90 に縮小したフレームで実測し、処理量が画素数に比例するとみなして式 14 でフルサイズへ換算した (この比例仮定の妥当性は後述のスケーリング実験で確認する)。
4. 方式 (d): `cv2.bitwise_not(frame)`。uint8 のビット反転 $\bar{p} = 255 - p$ を利用した OpenCV の既製実装である (前述のとおり参考計測のみ)。

$$T_{1280 \times 720} = T_{160 \times 90} \times \frac{1280 \times 720}{160 \times 90} = 64 T_{160 \times 90} \quad (14)$$

計測に先立ち、4 方式の出力を `np.array_equal` で比較し、すべて一致すること (方式 (c) は 160×90 で確認) を確認した。したがって以降の比較は「同一の結果を返す実装同士」の純粋な実行時間比較である。

5.8.2 ループ実装とベクトル化実装の比較

1 フレーム ($1280 \times 720 \times 3$, uint8) に対する各方式の実行時間を表 6 に、その対数軸での比較を図 14 に示す。

表 6: 色反転の実装方式別の 1 フレームあたり実行時間 (7 回計測の中央値・最小・最大)

方式	計測サイズ	中央値 [ms]	最小 [ms]	最大 [ms]	備考
(a) <code>255 - frame</code>	1280×720	2.216	0.691	2.525	採用手法
(b) <code>np.subtract(out=)</code>	1280×720	0.790	0.296	1.467	バッファ再利用
(c) Python 3 重ループ	160×90	39.98	20.97	87.62	実測
(c) Python 3 重ループ	1280×720	2558.9	1342.1	5607.5	64 倍換算値
(d) <code>cv2.bitwise_not</code>	1280×720	1.332	0.816	3.283	参考 (使用禁止)

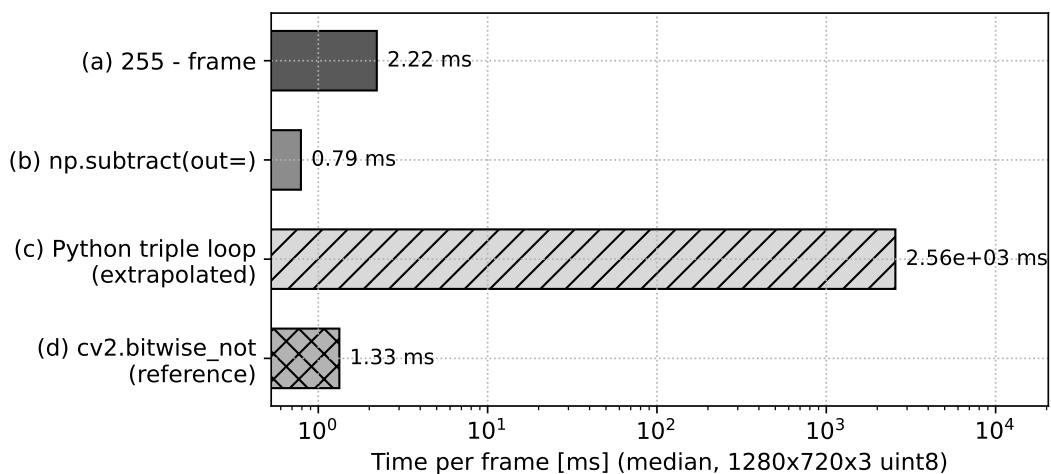


図 14: 実装方式別の 1 フレームあたり実行時間 (横軸は対数)。方式 (c) は 160×90 の実測値を式 14 で換算した推定値である。

表 6 のとおり、Python 3 重ループ (換算値 2559 ms) は採用手法の 255 - frame(2.216 ms) の約 1150 倍遅い。1 画素成分あたりに直すと、3 重ループは $39.98 \text{ ms} / (160 \times 90 \times 3) \approx 0.93 \mu\text{s}$ 、ベクトル演算は $2.216 \text{ ms} / (1280 \times 720 \times 3) \approx 0.80 \text{ ns}$ であり、3 桁の差がある。両者は式 13 という同一の計算をしているから、この差は減算そのものではなく、画素成分ごとに発生する Python インタープリタの処理 (バイトコードの解釈、配列要素の取り出しと uint8 スカラーオブジェクトの生成、ループ変数の管理) に費やされている。numpy のベクトル演算では、この要素ごとの解釈実行が配列全体で 1 回のコンパイル済み C ループに置き換わる。配列全体への一括操作 (ベクトル化) が配列プログラミングの性能の中心であることは NumPy の設計論文 [5] でも述べられており、さらに numpy は実行時に CPU の SIMD 命令セットを検出して最適化済みカーネルを選択する機構を持つ [6]。本実験の 3 桁の速度差は、これらの効果の実測値である。

換算値であるフレーム 1 枚 2.56 秒という時間は、動画の再生間隔 $1/29.97 \approx 33.4 \text{ ms}$ の約 77 倍であり、3 重ループ実装では 1 フレームの反転すら再生速度に間に合わない。一方、採用手法の 2.216 ms は再生間隔の 6.6 % にすぎない。課題の制約 (cv2.bitwise_not() 禁止) の下で numpy のベクトル演算を選択したことは、実行時間の観点から妥当であったといえる。

5.8.3 メモリ確保の影響と in-place 版・既製実装との比較

表 6 では、出力バッファを再利用する方式 (b) が 0.790 ms と、採用手法 (a) の 2.81 倍速い。両者の計算内容は同一で、異なるのは呼び出しごとに約 2.76 MB ($1280 \times 720 \times 3$ バイト) の出力配列を新規確保するかどうかだけである。したがってこの差は、大きな配列のメモリ確保・初期ページアクセスのコストが減算本体と同程度以上に大きいことを示している。同じことは計測の変動幅にも現れており、方式 (a) の 1 回あたり時間は最小 0.691 ms から最大 2.525 ms まで約 3.7 倍の幅を持つ。後述の表 7 の同一解像度 (1280×720) では方式 (a) が 0.691 ms と計測されており、これはメモリアロケータの状態 (解放済みページが再利用できるか) によって同一処理の時間が数倍変わるためと考えられる。

参考計測した cv2.bitwise_not(1.332 ms) は方式 (a) の 1.66 倍速いが、方式 (b) より遅い。また表 7 の同一解像度では逆に方式 (a) が方式 (d) の 2.7 倍速く、両者の大小関係は計測区間によって入れ替わる。すなわち方式 (a)・(b)・(d) の差はいずれもメモリ確保やアロケータ状態に起因する変動 (数倍) の範囲にあり、3 桁の差を持つループ実装との比較とは質的に異なる。この結果から、本課題の規模では「ベクトル化されているか否か」が支配的であり、使用が禁止されている cv2.bitwise_not を使えないことによる性能上の不利は実質的にないと結論できる。

なお、毎フレームの処理時間で概算すると、方式 (a) は入出力あわせて約 5.53 MB のデータを 2.216 ms で処理しており、実効メモリスループットは約 2.5 GB/s、方式 (b) では約 7.0 GB/s に達する。処理が演算律速ではなくメモリ律速に近いことも、メモリ確保の削減 (方式 (b)) がそのまま高速化につながる理由である。

5.8.4 画素数に対するスケーリング

色反転の処理量は画素数 N に比例するはずである。これを確認するため、解像度を 160×90 から 2560×1440 まで 4 倍刻みで変えて方式 (a) と方式 (d) の時間を計測した。処理時間が

$$T(N) = c N^\alpha \quad \Longleftrightarrow \quad \log_{10} T = \alpha \log_{10} N + \log_{10} c \quad (15)$$

に従うとすれば、両対数プロットの傾きが指数 α を与える。結果を表 7 と図 15 に示す。

最小二乗法による式 15 の傾きは、方式 (a) が $\alpha = 1.002$ 、方式 (d) が $\alpha = 1.009$ であり、いずれもほぼ 1 に一致した。図 15 でも両系列は傾き 1 の基準線とほぼ平行である。すなわち色反転の実行時間は画素数に対して線形 ($O(N)$) であり、方式 (c) の換算 (式 14) で用いた「処理量は画素数に比例する」という仮定はベクトル化実装において成立している。また、画素数が 256 倍 ($14400 \rightarrow 3686400$) 変わると時間は約

表 7: 解像度 (画素数) に対する 1 フレームあたり実行時間の変化 (中央値)

解像度	画素数	(a) 255 - frame [ms]	(d) cv2.bitwise_not [ms]
160×90	14400	0.0210	0.0174
320×180	57600	0.0793	0.0852
640×360	230400	0.2502	0.4770
1280×720	921600	0.6909	1.8703
2560×1440	3686400	7.4146	4.0265

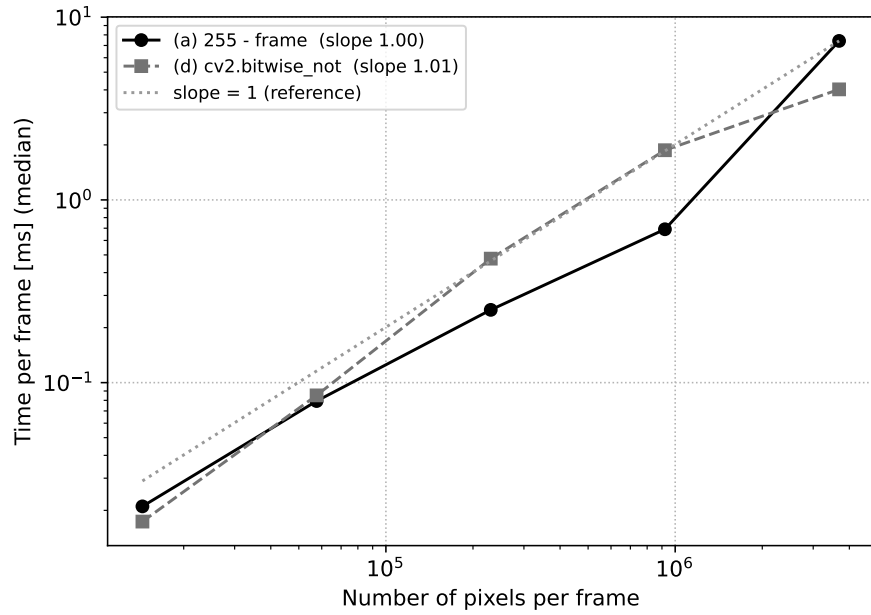


図 15: 画素数に対する色反転時間の両対数プロット。点線は傾き 1 (時間 $\propto$ 画素数) の基準線である。

350 倍変わるのに対し、同一画素数での方式 (a) と (d) の差は最大でも 2.7 倍にとどまる。解像度の選択が実装方式の選択より実行時間への影響はるかに大きいことが分かる。

5.8.5 動画処理全体に占める反転処理の割合

課題 4 の実処理は「読み込み → 反転 → 0.3 倍リサイズ → 書き出し」の繰り返しである。time.perf_counter で 4 区間を分けて累積計測した結果を表 8 と図 16 に示す。

表 8: 動画全体 (294 フレーム) の処理時間の内訳

区間	合計 [s]	1 フレーム平均 [ms]	割合 [%]
読み込み (cap.read)	5.292	18.00	67.7
反転 (255 - frame)	0.477	1.62	6.1
リサイズ (cv2.resize)	0.873	2.97	11.2
書き出し (writer.write)	1.172	3.99	15.0
4 区間合計	7.815	26.58	100.0

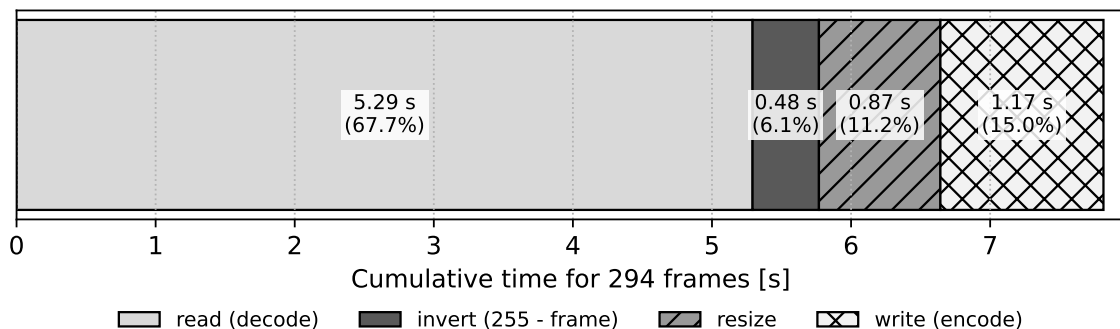


図 16: 動画全体 (294 フレーム) の処理時間の内訳 (積み上げ棒)。

表 8 のとおり、全体 7.81 秒のうち 67.7 % は動画のデコード (cap.read) が占め、反転処理はわずか 6.1 % (0.477 秒) である。1 フレームあたりの合計 26.6 ms は再生間隔 33.4 ms を下回っており、採用実装は実時間より速く動画を変換できる。

この内訳は 2 つの結論を与える。第一に、仮に反転処理を方式 (c) の 3 重ループで実装した場合、反転だけで $2.559 \text{ s} \times 294 \approx 752 \text{ 秒}$ を要し、全体の処理時間は約 7.8 秒から約 760 秒へと 97 倍に悪化すると見積もられる。ベクトル化の採否は、この処理系全体の実用性を左右する決定的な要因である。第二に、逆に反転処理をこれ以上高速化しても (たとえば方式 (b) や方式 (d) に置き換えて反転時間を 0 に近づけても) 全体は最大 6.1 % しか短縮されない。全体の律速はデコードであるから、さらなる高速化を図るなら反転の実装ではなく、フレームの読み飛ばしやデコード解像度の削減など入力側の工夫が必要である。採用した 255 - frame は、課題の制約を満たしつつ、処理全体の中でボトルネックにならない十分な速度を持つと結論できる。

参考文献

- [1] V. Tiwari and C. Bhatnagar: “A survey of recent work on video summarization: approaches and techniques,” Multimedia Tools and Applications, Vol. 80, pp. 27187–27221, 2021.
<https://link.springer.com/article/10.1007/s11042-021-10977-y>

- [2] E. Apostolidis, E. Adamantidou, A. I. Metsai, V. Mezaris and I. Patras: “Video Summarization Using Deep Neural Networks: A Survey,” Proceedings of the IEEE, 2021.
<https://arxiv.org/abs/2101.06072>
- [3] P. Virtanen, R. Gommers, T. E. Oliphant, et al.: “SciPy 1.0: fundamental algorithms for scientific computing in Python”, Nature Methods, Vol. 17, pp. 261–272 (2020).
<https://www.nature.com/articles/s41592-019-0686-2>
- [4] A. A. AlSabhany, A. H. Ali, F. Ridzuan, A. H. Azni and M. R. Mokhtar: “Digital audio steganography: Systematic review, classification, and analysis of the current state of the art”, Computer Science Review, Vol. 38, Article 100316 (2020). <https://doi.org/10.1016/j.cosrev.2020.100316>
- [5] C. R. Harris, K. J. Millman, S. J. van der Walt et al.: Array programming with NumPy, Nature, Vol. 585, pp. 357–362 (2020). <https://www.nature.com/articles/s41586-020-2649-2>
- [6] NumPy Developers: CPU/SIMD optimizations — NumPy v2.5 Manual (2026 年 7 月 2 日閲覧).
<https://numpy.org/doc/stable/reference/simd/index.html>

A 付録 A: プログラムソース

実際に実行したプログラム 6324034_final_exam.py を無加工で示す。

```
# =====
# 計算機科学応用演習 final exam
# 6324034 木村 竜輝
#
# 課題 1: 動画の読み込み (サイズ・FPS・総フレーム数)
# 課題 2: サムネイル画像の作成 (代表フレーム選択)
# 課題 3: ファイル名のパース (タイトルと ID)
# 課題 4: 色反転と動画の保存 (0.3 倍リサイズ)
# 課題 5: 音声の可視化 (スペクトログラム)
# 課題 6: n-gram(2-gram / stop words 除去)
# 課題 7: 動画の変わり目検出 (フレーム間差分)
# 課題 8: オブジェクト指向 (Video クラス・高階関数)
# =====

import os
import re
from collections import Counter

import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy import signal

# -----
# パス設定
# 課題指定の Colab パスを既定とし、環境変数 DATASET_ROOT があれば
# ローカル実行用に差し替える (Colab では未設定なので指定パスになる)
# -----
DATASET_ROOT = os.environ.get(
    "DATASET_ROOT", "/content/drive/MyDrive/Colab Notebooks"
)
VIDEO_DIR = os.path.join(DATASET_ROOT, "dataset", "videos")
AUDIO_DIR = os.path.join(DATASET_ROOT, "dataset", "audio")
```

```

TEXT_DIR = os.path.join(DATASET_ROOT, "dataset", "text")
VIDEO_OUT_DIR = os.path.join(VIDEO_DIR, "output")
IMAGE_OUT_DIR = os.path.join(DATASET_ROOT, "dataset", "images", "output")

STUDENT_ID = "6324034"

PEOPLE_VIDEO = os.path.join(VIDEO_DIR, "People - 84973.mp4")
CAT_VIDEO = os.path.join(VIDEO_DIR, "Cat - 66004.mp4")
DETECTION_VIDEO = os.path.join(VIDEO_DIR, "Detection.mp4")
REPORT_AUDIO = os.path.join(AUDIO_DIR, "report_audio.wav")
TEXT_ENGLISH = os.path.join(TEXT_DIR, "text_english.txt")

# =====
# 課題 1: 動画の読み込み
# =====
def task1_video_info(video_path):
    """動画を読み込み、縦・横のサイズ、FPS、総フレーム数を出力する。"""
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"動画を開けません: {video_path}")
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    cap.release()

    print(f"横のサイズ: {width}")
    print(f"縦のサイズ: {height}")
    print(f"FPS: {fps}")
    print(f"総フレーム数: {frame_count}")
    return width, height, fps, frame_count

# =====
# 課題 2: サムネイル画像の作成
# 中央フレームではなく「代表フレーム」を自作の基準で選ぶ:
# 動画全体の平均ヒストグラムに最も近いヒストグラムを持つフレーム
# =====
def compute_gray_histogram(frame, bins=64):
    """フレームのグレースケール正規化ヒストグラムを返す。"""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    hist, _ = np.histogram(gray, bins=bins, range=(0, 256))
    hist = hist.astype(np.float64)
    total = hist.sum()
    if total > 0:
        hist = hist / total
    return hist

def task2_thumbnail(video_path, out_path, sample_step=5):
    """動画の代表フレームをサムネイルとして保存する。

    全フレームを sample_step 間隔でサンプリングし、各フレームの
    グレースケールヒストグラムを計算する。全サンプルの平均ヒスト
    グラムに L2 距離が最も近いフレームを「動画全体を最もよく代表
    するフレーム」として選択する。
    """
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"動画を開けません: {video_path}")

```

```

histograms = []
frame_indices = []
index = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    if index % sample_step == 0:
        histograms.append(compute_gray_histogram(frame))
        frame_indices.append(index)
    index += 1

hist_array = np.array(histograms)
mean_hist = hist_array.mean(axis=0)
distances = np.linalg.norm(hist_array - mean_hist, axis=1)
best_index = frame_indices[int(np.argmin(distances))]

cap.set(cv2.CAP_PROP_POS_FRAMES, best_index)
ret, best_frame = cap.read()
cap.release()
if not ret:
    raise IOError(f"フレーム {best_index} を読み込めません")

cv2.imwrite(out_path, best_frame)
print(f"代表フレーム番号: {best_index}")
print(f"サムネイルを保存しました: {out_path}")
return best_index, best_frame

# =====
# 課題 3: ファイル名のパース
# =====
def task3_parse_filename(filename):
    """ファイル名からタイトルと ID を取り出す。

    例: "People - 84973.mp4" -> ("People", "84973")
    拡張子を除いた後、右端の " - " で 1 回だけ分割することで、
    タイトル側に "-" が含まれる場合にも対応する。
    """
    stem = os.path.splitext(os.path.basename(filename))[0]
    parts = stem.rsplit(" - ", 1)
    if len(parts) != 2:
        raise ValueError(f"' タイトル - ID' 形式ではありません: {filename}")
    title = parts[0].strip()
    video_id = parts[1].strip()
    print(f"Title: {title}")
    print(f"ID: {video_id}")
    return title, video_id

# =====
# 課題 4: 色反転と動画の保存
# =====
def task4_reverse_video(video_path, out_path, scale=0.3):
    """各フレームの色を反転し、縦横 scale 倍にした動画を保存する。

    反転後の画素値 = 画素値の最大値 (255) - 元の画素値
    cv2.bitwise_not() は使わず、numpy の配列演算で計算する。
    """
    cap = cv2.VideoCapture(video_path)

```

```

if not cap.isOpened():
    raise IOError(f"動画を開けません: {video_path}")

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
new_size = (int(width * scale), int(height * scale))

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(out_path, fourcc, fps, new_size)

frame_count = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    reversed_frame = 255 - frame # uint8 の最大値から引く
    resized = cv2.resize(reversed_frame, new_size)
    writer.write(resized)
    frame_count += 1

cap.release()
writer.release()
print(f"色反転動画を保存しました: {out_path}")
print(f"出力サイズ: {new_size[0]}x{new_size[1]}, フレーム数: {frame_count}")
return out_path

# =====
# 課題 5: 音声の可視化 (スペクトログラム)
# =====
def task5_spectrogram(audio_path, out_path, nperseg=1024, noverlap=768,
                      window="hann", vmin_db=-100, cmap="gray"):
    """音声のスペクトログラムを描画して保存する。

    STFT のパラメータ (窓長 nperseg、オーバーラップ noverlap、窓関数)
    を引数で調整できるようにし、時間-周波数領域に隠されたメッセージを
    可視化する。強度は dB スケールで表示する。
    """
    rate, data = wavfile.read(audio_path)
    if data.ndim > 1: # ステレオならモノラル化
        data = data.mean(axis=1)
    data = data.astype(np.float64)

    freqs, times, sxx = signal.spectrogram(
        data, fs=rate, window=window, nperseg=nperseg, noverlap=noverlap
    )
    sxx_db = 10 * np.log10(sxx + 1e-12)

    fig, ax = plt.subplots(figsize=(10, 5))
    mesh = ax.pcolormesh(times, freqs, sxx_db, shading="auto",
                        cmap=cmap, vmin=vmin_db)
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("Frequency [Hz]")
    ax.set_title(f"Spectrogram (nperseg={nperseg}, noverlap={noverlap}, "
                f"window={window})")
    fig.colorbar(mesh, ax=ax, label="Power [dB]")
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"サンプリングレート: {rate} Hz, サンプル数: {len(data)}")

```



```

print(f"スペクトログラムを保存しました: {out_path}")
return rate, len(data)

# =====
# 課題 6: n-gram
# =====
# 6-b で用いる stop words(冠詞・be 動詞・前置詞・接続詞・代名詞など、
# 文の内容よりも文法機能を担う高頻度語を自分で設計した)
STOP_WORDS = {
    "the", "a", "an", "is", "are", "was", "were", "be", "been", "being",
    "of", "to", "in", "and", "or", "but", "on", "at", "by", "for",
    "with", "from", "as", "that", "this", "these", "those", "it", "its",
    "he", "she", "they", "them", "his", "her", "their", "we", "you",
    "i", "my", "your", "our", "not", "no", "so", "if", "than", "then",
    "there", "here", "have", "has", "had", "do", "does", "did", "will",
    "would", "can", "could", "shall", "should", "may", "might", "must",
    "am", "into", "up", "down", "out", "about", "over", "under", "all",
    "any", "each", "which", "who", "whom", "what", "when", "where",
    "how", "why", "s", "t",
}

def tokenize(text):
    """テキストを小文字化し、英単語のみを取り出す。"""
    return re.findall(r"[a-z]+", text.lower())

def build_2grams(words):
    """単語列から連続する 2 語の組 (2-gram) のリストを作る。"""
    return [f"{words[i]} {words[i + 1]}" for i in range(len(words) - 1)]

def plot_top_2grams(counter, top_n, out_path, title):
    """2-gram の出現頻度上位 top_n 個を棒グラフとして保存する。"""
    top = counter.most_common(top_n)
    labels = [pair for pair, _ in top]
    values = [count for _, count in top]

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.bar(range(len(labels)), values, color="gray", edgecolor="black")
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels(labels, rotation=45, ha="right")
    ax.set_xlabel("2-gram")
    ax.set_ylabel("Frequency")
    ax.set_title(title)
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    for pair, count in top:
        print(f"{pair}: {count}")
    print(f"グラフを保存しました: {out_path}")
    return top

def task6a_2gram(text_path, out_path, top_n=10):
    """テキストの 2-gram 出現頻度上位 top_n をグラフ化する。"""
    with open(text_path, encoding="utf-8") as f:
        text = f.read()
    words = tokenize(text)
    counter = Counter(build_2grams(words))

```

```

print(f"総単語数: {len(words)}, 異なり 2-gram 数: {len(counter)}")
return plot_top_2grams(counter, top_n, out_path,
                        "Top 2-grams (with stop words)")

def task6b_2gram_no_stopwords(text_path, out_path, top_n=10):
    """stop words を除去した 2-gram 出現頻度上位 top_n をグラフ化する。"""
    with open(text_path, encoding="utf-8") as f:
        text = f.read()
    words = [w for w in tokenize(text) if w not in STOP_WORDS]
    counter = Counter(build_2grams(words))
    print(f"stop words 除去後の単語数: {len(words)}, "
          f"異なり 2-gram 数: {len(counter)}")
    return plot_top_2grams(counter, top_n, out_path,
                          "Top 2-grams (stop words removed)")

# =====
# 課題 7: 動画の変わり目検出
# =====
def task7_frame_differences(video_path):
    """全フレームについて前フレームとの画素値変化量を計算する。

    1. 各フレームをグレースケールにする
    2. t 番目と t-1 番目の各画素の差の絶対値を計算する
       (absdiff() は使わず、符号付き整数に変換して numpy で計算)
    3. 全画素の差を足し合わせて変化量とする
    """
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"動画を開けません: {video_path}")

    diffs = []
    prev_gray = None
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).astype(np.int32)
        if prev_gray is not None:
            diffs.append(int(np.abs(gray - prev_gray).sum()))
        prev_gray = gray
    cap.release()
    # diffs[t-1] が「フレーム t-1 と t の間の変化量」
    return np.array(diffs, dtype=np.float64)

def task7a_plot_differences(diffs, out_path):
    """横軸フレーム番号、縦軸変化量のグラフを描画する。"""
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(np.arange(1, len(diffs) + 1), diffs, color="black", lw=0.8)
    ax.set_xlabel("Frame index")
    ax.set_ylabel("Sum of absolute differences")
    ax.set_title("Inter-frame change")
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"変化量グラフを保存しました: {out_path}")

def detect_by_mean_std(diffs, k=3.0):

```

```

    """閾値 = 平均 + k × 標準偏差 で変わり目を検出する。"""
    threshold = diffs.mean() + k * diffs.std()
    frames = np.where(diffs > threshold)[0] + 1 # diffs[i] はフレーム i+1
    return threshold, frames

def detect_by_top_peaks(diffs, n_peaks=3):
    """変化量の上位 n_peaks 個のフレームを変わり目とする。"""
    order = np.argsort(diffs)[::-1][:n_peaks]
    return np.sort(order + 1)

def detect_by_moving_average(diffs, window=15, k=4.0):
    """移動平均で平滑化した基準線から大きく外れたフレームを検出する。

    各フレームの変化量を、その近傍 window フレームの移動平均と比較し、
    移動平均の k 倍を超えたフレームを変わり目とする。
    """
    kernel = np.ones(window) / window
    baseline = np.convolve(diffs, kernel, mode="same")
    frames = np.where(diffs > k * baseline)[0] + 1
    return frames

def save_boundary_frames(video_path, frame_indices, out_dir, prefix):
    """検出したフレームとその前フレームの画像を保存して目視確認する。"""
    cap = cv2.VideoCapture(video_path)
    saved = []
    for fi in frame_indices:
        for offset in (-1, 0):
            target = fi + offset
            cap.set(cv2.CAP_PROP_POS_FRAMES, target)
            ret, frame = cap.read()
            if ret:
                path = os.path.join(out_dir, f"{prefix}_frame{target}.jpg")
                cv2.imwrite(path, frame)
                saved.append(path)
    cap.release()
    print(f"確認用フレーム画像を {len(saved)} 枚保存しました")
    return saved

# =====
# 課題 8: オブジェクト指向 (Video クラス・高階関数)
# =====
def invert_frame(frame):
    """フレームの色を反転する (課題 4 と同じ処理)。"""
    return 255 - frame

def grayscale_frame(frame):
    """フレームをグレースケール化する (動画保存用に 3ch へ戻す)。"""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    return cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

def binarize_frame(frame, threshold=128):
    """フレームを二値化する (動画保存用に 3ch へ戻す)。"""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    binary = np.where(gray >= threshold, 255, 0).astype(np.uint8)
    return cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)

```

```

class Video:
    """動画 1 本を表すクラス。

    インスタンス変数:
        path        -- 動画ファイルのパス
        title       -- ファイル名から取り出したタイトル
        video_id    -- ファイル名から取り出した ID
        thumbnail   -- 代表フレームの画像 (numpy 配列)
    """

    def __init__(self, path):
        self.path = path
        self.title = None
        self.video_id = None
        self.thumbnail = None

    # --- 課題 3 をメソッド化 ---
    def parse_filename(self):
        """ファイル名からタイトルと ID を取り出してインスタンス変数に代入する。"""
        stem = os.path.splitext(os.path.basename(self.path))[0]
        parts = stem.rsplit("-", 1)
        if len(parts) != 2:
            raise ValueError(f"タイトル - ID 形式ではありません: {self.path}")
        self.title = parts[0].strip()
        self.video_id = parts[1].strip()
        return self.title, self.video_id

    # --- 課題 2 をメソッド化 ---
    def create_thumbnail(self, sample_step=5):
        """代表フレームを選んでインスタンス変数 thumbnail に代入する。"""
        best_index, best_frame = task2_thumbnail_frame(
            self.path, sample_step=sample_step
        )
        self.thumbnail = best_frame
        return best_index

    def get_info(self):
        """4 つのインスタンス変数を辞書型で返す。"""
        return {
            "path": self.path,
            "title": self.title,
            "video_id": self.video_id,
            "thumbnail": self.thumbnail,
        }

    # --- 課題 4 をメソッド化 (高階関数版) ---
    def process_video(self, frame_func, out_path, scale=0.3):
        """フレーム処理関数 frame_func を全フレームに適用した動画を作る。

        frame_func: フレーム (numpy 配列) を受け取り、同じ大きさの
                     フレームを返す関数。二値化・グレースケール化・
                     色反転などを差し替えるだけで動画処理ができる。
        """
        cap = cv2.VideoCapture(self.path)
        if not cap.isOpened():
            raise IOError(f"動画を開けません: {self.path}")
        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        fps = cap.get(cv2.CAP_PROP_FPS)

```

```

        new_size = (int(width * scale), int(height * scale))

        fourcc = cv2.VideoWriter_fourcc(*"mp4v")
        writer = cv2.VideoWriter(out_path, fourcc, fps, new_size)
        while True:
            ret, frame = cap.read()
            if not ret:
                break
            processed = frame_func(frame)
            writer.write(cv2.resize(processed, new_size))
        cap.release()
        writer.release()
        return out_path

    def create_inverse_video(self, out_path, scale=0.3):
        """色反転動画を作成する (process_video に invert_frame を渡す)。"""
        return self.process_video(invert_frame, out_path, scale=scale)

def task2_thumbnail_frame(video_path, sample_step=5):
    """課題 2 の代表フレーム選択 (フレームを返す内部版)。"""
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"動画を開けません: {video_path}")
    histograms = []
    frame_indices = []
    index = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if index % sample_step == 0:
            histograms.append(compute_gray_histogram(frame))
            frame_indices.append(index)
        index += 1
    hist_array = np.array(histograms)
    mean_hist = hist_array.mean(axis=0)
    distances = np.linalg.norm(hist_array - mean_hist, axis=1)
    best_index = frame_indices[int(np.argmin(distances))]
    cap.set(cv2.CAP_PROP_POS_FRAMES, best_index)
    ret, best_frame = cap.read()
    cap.release()
    if not ret:
        raise IOError(f"フレーム {best_index} を読み込みません")
    return best_index, best_frame

# =====
# メイン処理
# =====
def main():
    os.makedirs(VIDEO_OUT_DIR, exist_ok=True)
    os.makedirs(IMAGE_OUT_DIR, exist_ok=True)

    print("=" * 60)
    print("課題 1: 動画の読み込み")
    print("=" * 60)
    task1_video_info(PEOPLE_VIDEO)

    print()
    print("=" * 60)

```

```

print("課題 2: サムネイル画像の作成")
print("=" * 60)
thumb_path = os.path.join(IMAGE_OUT_DIR, f"{STUDENT_ID}_thumbnail.jpg")
task2_thumbnail(PEOPLE_VIDEO, thumb_path)

print()
print("=" * 60)
print("課題 3: ファイル名のパース")
print("=" * 60)
task3_parse_filename("People - 84973.mp4")
task3_parse_filename("Cat - 66004.mp4")

print()
print("=" * 60)
print("課題 4: 色反転と動画の保存")
print("=" * 60)
reverse_path = os.path.join(VIDEO_OUT_DIR, f"{STUDENT_ID}_cat_reverse.mp4")
task4_reverse_video(CAT_VIDEO, reverse_path, scale=0.3)

print()
print("=" * 60)
print("課題 5: 音声の可視化")
print("=" * 60)
spec_path = os.path.join(IMAGE_OUT_DIR, f"{STUDENT_ID}_spectrogram.jpg")
task5_spectrogram(REPORT_AUDIO, spec_path)

print()
print("=" * 60)
print("課題 6-a: 2-gram")
print("=" * 60)
gram_path = os.path.join(IMAGE_OUT_DIR, f"{STUDENT_ID}_2gram_graph.jpg")
task6a_2gram(TEXT_ENGLISH, gram_path)

print()
print("=" * 60)
print("課題 6-b: stop words 除去")
print("=" * 60)
gram_sw_path = os.path.join(
    IMAGE_OUT_DIR, f"{STUDENT_ID}_2gram_graph_no_stopwords.jpg"
)
task6b_2gram_no_stopwords(TEXT_ENGLISH, gram_sw_path)

print()
print("=" * 60)
print("課題 7: 動画の変わり目検出")
print("=" * 60)
diffs = task7_frame_differences(DETECTION_VIDEO)
print(f"フレーム間変化量の数: {len(diffs)}")
diff_plot_path = os.path.join(IMAGE_OUT_DIR, f"{STUDENT_ID}_framediff.jpg")
task7a_plot_differences(diffs, diff_plot_path)

print("--- 方法 1: 平均 + 3 × 標準偏差 ---")
threshold, frames_std = detect_by_mean_std(diffs, k=3.0)
print(f"閾値: {threshold:.1f}")
print(f"検出フレーム: {frames_std.tolist()}")

print("--- 方法 2: 変化量上位 3 ピーク ---")
frames_peak = detect_by_top_peaks(diffs, n_peaks=3)
print(f"検出フレーム: {frames_peak.tolist()}")

print("--- 方法 3: 移動平均基準 ---")

```

```

frames_ma = detect_by_moving_average(diffs, window=15, k=4.0)
print(f"検出フレーム: {frames_ma.tolist()}")

save_boundary_frames(DETECTION_VIDEO, frames_peak, IMAGE_OUT_DIR,
                     f"{STUDENT_ID}_boundary")

print()
print("=" * 60)
print("課題 8: Video クラスの動作確認")
print("=" * 60)
video = Video(PEOPLE_VIDEO)
video.parse_filename()
thumb_index = video.create_thumbnail()
info = video.get_info()
print(f"path: {info['path']}")
print(f"title: {info['title']}")
print(f"video_id: {info['video_id']}")
print(f"thumbnail: フレーム {thumb_index}, "
      f"形状 {info['thumbnail'].shape}")

cat = Video(CAT_VIDEO)
cat.parse_filename()
print("--- 高階関数による動画処理 ---")
for func, name in [(binarize_frame, "binary"),
                  (grayscale_frame, "gray"),
                  (invert_frame, "reverse_hof")]:
    out = os.path.join(VIDEO_OUT_DIR, f"{STUDENT_ID}_cat_{name}.mp4")
    cat.process_video(func, out, scale=0.3)
    print(f"{name}: {out}")

if __name__ == "__main__":
    main()

```

B 付録 B: 実行結果 (全文)

プログラムの実行出力を無加工で示す。

```

=====
課題 1: 動画の読み込み
=====
横のサイズ: 1280
縦のサイズ: 720
FPS: 25.0
総フレーム数: 241

=====
課題 2: サムネイル画像の作成
=====
代表フレーム番号: 185
サムネイルを保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/images/output/6324034_thumbnail.

=====
課題 3: ファイル名のパース
=====
Title: People
ID: 84973
Title: Cat
ID: 66004

```

```

=====
課題 4: 色反転と動画の保存
=====
色反転動画を保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/output/6324034_cat_revers
出力サイズ: 384x216, フレーム数: 294

=====
課題 5: 音声の可視化
=====
サンプリングレート: 44100 Hz, サンプル数: 264600
スペクトログラムを保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/images/output/6324034_spectr

=====
課題 6-a: 2-gram
=====
総単語数: 984, 異なり 2-gram 数: 858
natural language: 15
language processing: 9
in the: 9
e g: 9
of the: 8
the s: 7
as a: 5
machine translation: 5
with the: 4
nlp is: 3
グラフを保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/images/output/6324034_2gram_graph.j

=====
課題 6-b: stop words 除去
=====
stop words 除去後の単語数: 614, 異なり 2-gram 数: 560
natural language: 15
language processing: 9
e g: 9
machine translation: 5
language understanding: 3
machine learning: 3
artificial intelligence: 2
language data: 2
generation natural: 2
symbolic nlp: 2
グラフを保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/images/output/6324034_2gram_graph.n

=====
課題 7: 動画の変わり目検出
=====
フレーム間変化量の数: 321
変化量グラフを保存しました: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/images/output/6324034_framedif
--- 方法 1: 平均 + 3 × 標準偏差 ---
閾値: 71682248.1
検出フレーム: [78, 214, 257]
--- 方法 2: 変化量上位 3 ピーク ---
検出フレーム: [78, 214, 257]
--- 方法 3: 移動平均基準 ---
検出フレーム: [78, 214, 257]
確認用フレーム画像を 6 枚保存しました

=====
課題 8: Video クラスの動作確認
=====

```



```

=====
path: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/People - 84973.mp4
title: People
video_id: 84973
thumbnail: フレーム 185, 形状 (720, 1280, 3)
--- 高階関数による動画処理 ---
binary: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/output/6324034_cat_binary.mp4
gray: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/output/6324034_cat_gray.mp4
reverse_hof: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/output/6324034_cat_reverse_hof.mp4

```

C 付録 C: 追加実験のソースと出力

考察で行った追加実験のソースコードと実行出力を無加工で示す。

C.1 実験ソース: exp01_thumbnail.py

サムネイル選択手法の定量比較実験 (考察参照) に用いたスクリプトを無加工で示す。

```

# =====
# exp01_thumbnail.py
# 課題 2 のサムネイル (代表フレーム) 選択手法の定量比較実験
#
# 比較する 4 手法:
# (a) 中央フレーム法      : フレーム番号 N//2 を選ぶ
# (b) 平均ヒストグラム法  : 採用手法。グレースケール 64 bin 正規化
#                           ヒストグラムの平均への L2 距離最小
#                           (src/6324034_final_exam.py の
#                           task2_thumbnail_frame と同一ロジック,
#                           sample_step=5)
# (c) 鮮明度法            : ラプラシアン分散が最大のフレーム
# (d) カラフルネス法      : Hasler-Suesstrunk 指標が最大のフレーム
#
# 出力:
# experiments/data/exp01_traces.csv   : 全フレームの各評価値の推移
# experiments/data/exp01_summary.csv  : 各手法の選択結果・計算時間
# experiments/figures/exp01_metric_traces.pdf : 評価値推移 (2x2)
# experiments/figures/exp01_normalized.pdf   : 正規化評価値の重ね描き
# experiments/figures/exp01_selected_frames.pdf : 選択フレーム画像 (2x2)
#
# 実行方法:
# cd "/Users/kimura2003/Downloads/rikadai_kadai_python"
# DATASET_ROOT="$PWD" MPLBACKEND=Agg python3 experiments/exp01_thumbnail.py
# =====
import csv
import os
import time

import cv2
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

DATASET_ROOT = os.environ.get("DATASET_ROOT", ".")
VIDEO_PATH = os.path.join(DATASET_ROOT, "dataset", "videos",
                           "People - 84973.mp4")
DATA_DIR = os.path.join(DATASET_ROOT, "experiments", "data")
FIG_DIR = os.path.join(DATASET_ROOT, "experiments", "figures")

```

```

HIST_BINS = 64      # 採用手法と同じビン数
SAMPLE_STEP = 5     # 採用手法と同じサンプリング間隔

# -----
# 評価指標
# -----
def compute_gray_histogram(frame, bins=HIST_BINS):
    """グレースケール正規化ヒストグラム (採用手法と同一ロジック)。"""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    hist, _ = np.histogram(gray, bins=bins, range=(0, 256))
    hist = hist.astype(np.float64)
    total = hist.sum()
    if total > 0:
        hist = hist / total
    return hist

def laplacian_variance(frame):
    """鮮明度指標: グレースケール画像のラプラシアン分散。"""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    lap = cv2.Laplacian(gray, cv2.CV_64F)
    return float(lap.var())

def colorfulness_hs(frame):
    """Hasler-Suesstrunk のカラフルネス指標  $M_0$ 。

     $rg = R - G$ ,  $y_b = (R + G)/2 - B$  として
     $M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3 * \sqrt{\mu_{rg}^2 + \mu_{yb}^2}$ 
    """
    b = frame[:, :, 0].astype(np.float64)
    g = frame[:, :, 1].astype(np.float64)
    r = frame[:, :, 2].astype(np.float64)
    rg = r - g
    yb = 0.5 * (r + g) - b
    sigma = np.sqrt(rg.var() + yb.var())
    mu = np.sqrt(rg.mean() ** 2 + yb.mean() ** 2)
    return float(sigma + 0.3 * mu)

def percentile_rank(values, v):
    """values の中で v 以下の値が占める割合 (百分率)。"""
    values = np.asarray(values, dtype=np.float64)
    return float(100.0 * np.mean(values <= v))

def spearman_corr(x, y):
    """Spearman 順位相関係数 (順位化してから Pearson)。"""
    rx = np.argsort(np.argsort(x)).astype(np.float64)
    ry = np.argsort(np.argsort(y)).astype(np.float64)
    return float(np.corrcoef(rx, ry)[0, 1])

# -----
# メイン処理
# -----
def main():
    os.makedirs(DATA_DIR, exist_ok=True)
    os.makedirs(FIG_DIR, exist_ok=True)

```

```

print("=" * 70)
print("exp01: サムネイル選択手法の定量比較")
print("=" * 70)
print(f"OpenCV {cv2.__version__} / NumPy {np.__version__} / "
      f"matplotlib {matplotlib.__version__}")
print(f"video: {VIDEO_PATH}")

cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise IOError(f"動画を開けません: {VIDEO_PATH}")
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
n_frames_prop = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(f"size: {width}x{height}, fps: {fps}, frames(prop): {n_frames_prop}")

# --- 1 パスで全フレームの各評価値と計算時間を収集 -----
hists = []
lap_vars = []
colorfulness = []
t_hist = []
t_lap = []
t_col = []
t_decode = 0.0

index = 0
while True:
    t0 = time.perf_counter()
    ret, frame = cap.read()
    t_decode += time.perf_counter() - t0
    if not ret:
        break

    t0 = time.perf_counter()
    hists.append(compute_gray_histogram(frame))
    t_hist.append(time.perf_counter() - t0)

    t0 = time.perf_counter()
    lap_vars.append(laplacian_variance(frame))
    t_lap.append(time.perf_counter() - t0)

    t0 = time.perf_counter()
    colorfulness.append(colorfulness_hs(frame))
    t_col.append(time.perf_counter() - t0)

    index += 1
cap.release()

n_frames = index
print(f"frames(read): {n_frames}")
print(f"decode time total: {t_decode:.4f} s")

hists = np.array(hists)
lap_vars = np.array(lap_vars)
colorfulness = np.array(colorfulness)
frames_axis = np.arange(n_frames)

# --- (a) 中央フレーム法 -----
t0 = time.perf_counter()
center_index = n_frames // 2
time_center = time.perf_counter() - t0

```

```

center_dist = np.abs(frames_axis - center_index).astype(np.float64)

# --- (b) 平均ヒストグラム法 (採用手法, sample_step=5) -----
t0 = time.perf_counter()
sampled_idx = np.arange(0, n_frames, SAMPLE_STEP)
sampled_hists = hists[sampled_idx]
mean_hist_s5 = sampled_hists.mean(axis=0)
dists_s5 = np.linalg.norm(sampled_hists - mean_hist_s5, axis=1)
hist_index_s5 = int(sampled_idx[int(np.argmin(dists_s5))])
time_hist_select = time.perf_counter() - t0
time_hist = float(np.sum(np.array(t_hist)[sampled_idx])) + time_hist_select

# 参考: sample_step=1(全フレーム) での平均ヒストグラム法
mean_hist_s1 = hists.mean(axis=0)
dists_s1 = np.linalg.norm(hists - mean_hist_s1, axis=1)
hist_index_s1 = int(np.argmin(dists_s1))

# --- (c) 鮮明度法 (ラプラシアン分散最大) -----
t0 = time.perf_counter()
lap_index = int(np.argmax(lap_vars))
time_lap = float(np.sum(t_lap)) + (time.perf_counter() - t0)

# --- (d) カラフルネス法 (Hasler-Suesstrunk 指標最大) -----
t0 = time.perf_counter()
col_index = int(np.argmax(colorfulness))
time_col = float(np.sum(t_col)) + (time.perf_counter() - t0)

methods = [
    ("center", "中央フレーム法", center_index, time_center),
    ("hist", "平均ヒストグラム法 (採用)", hist_index_s5, time_hist),
    ("laplacian", "鮮明度法", lap_index, time_lap),
    ("colorfulness", "カラフルネス法", col_index, time_col),
]

print()
print("---- 各手法の選択フレームと計算時間 ----")
for key, name, sel, t in methods:
    print(f"{key:14s} {name:22s} frame={sel:4d} time={t:.6f} s")
print(f"(参考) 平均ヒストグラム法 sample_step=1: frame={hist_index_s1}")
print(f"(参考) sample_step=5 と step=1 の選択差: "
      f"{abs(hist_index_s1 - hist_index_s5)} フレーム")

# --- 各指標の基本統計 -----
print()
print("---- 各評価値の基本統計 (全フレーム) ----")
stats_rows = [
    ("hist_l2_dist(step=1)", dists_s1),
    ("laplacian_var", lap_vars),
    ("colorfulness", colorfulness),
]
for name, arr in stats_rows:
    print(f"{name:22s} min={arr.min():12.6g} max={arr.max():12.6g} "
          f"mean={arr.mean():12.6g} std={arr.std():12.6g}")

# --- 指標間の相互評価 (クロス評価表) -----
print()
print("---- 各手法の選択フレームにおける他指標の値と百分位 ----")
print("(百分位はその値以下のフレームが占める割合 [%]。)"
      "hist_l2 は小さいほど良い / lap_var, colorfulness は大きいほど良い")
header = (f"{'method':14s} {'frame':>5s} "
          f"{'hist_l2':>10s} {'pct':>6s} ")

```

```

        f"{'lap_var':>10s} {'pct':>6s} "
        f"{'colorful':>10s} {'pct':>6s}")
print(header)
cross_rows = []
for key, name, sel, t in methods:
    h = dists_s1[sel]
    l = lap_vars[sel]
    c = colorfulness[sel]
    ph = percentile_rank(dists_s1, h)
    pl = percentile_rank(lap_vars, l)
    pc = percentile_rank(colorfulness, c)
    print(f"{key:14s} {sel:5d} "
          f"{h:10.6f} {ph:6.1f} "
          f"{l:10.2f} {pl:6.1f} "
          f"{c:10.2f} {pc:6.1f}")
    cross_rows.append([key, sel, h, ph, l, pl, c, pc, t])

# --- 指標間の相関 -----
print()
print("---- 指標間の相関係数 (全フレーム, Pearson / Spearman) ----")
pairs = [
    ("hist_l2_dist vs laplacian_var", dists_s1, lap_vars),
    ("hist_l2_dist vs colorfulness", dists_s1, colorfulness),
    ("laplacian_var vs colorfulness", lap_vars, colorfulness),
]
for name, x, y in pairs:
    pear = float(np.corrcoef(x, y)[0, 1])
    spear = spearman_corr(x, y)
    print(f"{name:32s} Pearson={pear:+.4f} Spearman={spear:+.4f}")

# --- CSV 出力 -----
traces_csv = os.path.join(DATA_DIR, "exp01_traces.csv")
with open(traces_csv, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["frame", "center_dist", "hist_l2_dist_step1",
                "laplacian_var", "colorfulness"])
    for i in range(n_frames):
        w.writerow([i, center_dist[i], f"{dists_s1[i]:.8f}",
                    f"{lap_vars[i]:.6f}", f"{colorfulness[i]:.6f}"])
print()
print(f"saved: {traces_csv}")

summary_csv = os.path.join(DATA_DIR, "exp01_summary.csv")
with open(summary_csv, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["method", "selected_frame",
                "hist_l2_dist", "hist_l2_percentile",
                "laplacian_var", "laplacian_percentile",
                "colorfulness", "colorfulness_percentile",
                "time_sec"])
    for row in cross_rows:
        w.writerow([row[0], row[1],
                    f"{row[2]:.8f}", f"{row[3]:.1f}",
                    f"{row[4]:.4f}", f"{row[5]:.1f}",
                    f"{row[6]:.4f}", f"{row[7]:.1f}",
                    f"{row[8]:.6f}"])
print(f"saved: {summary_csv}")

# --- 図 1: 評価値の推移 (2x2 サブプロット) -----
plt.rcParams.update({"font.size": 9})
fig, axes = plt.subplots(2, 2, figsize=(8.0, 5.6))

```

```

panels = [
    (axes[0, 0], center_dist, center_index,
     "(a) Center-frame:  $|i - 120|$ ",
     "distance from center (frames)"),
    (axes[0, 1], dists_s1, hist_index_s5,
     "(b) Mean-histogram (adopted): L2 distance",
     "L2 distance to mean histogram (-)"),
    (axes[1, 0], lap_vars, lap_index,
     "(c) Sharpness: Laplacian variance",
     "Laplacian variance ((gray level) $^2$ "),
    (axes[1, 1], colorfulness, col_index,
     "(d) Colorfulness: Hasler-Suesstrunk  $MM$ ",
     "colorfulness  $MM$  (-)"),
]
for ax, arr, sel, title, ylabel in panels:
    ax.plot(frames_axis, arr, color="black", linewidth=0.9)
    ax.axvline(sel, color="black", linestyle=(0, (5, 2)), linewidth=1.2)
    ymin, ymax = ax.get_ylim()
    ax.text(sel + 3, ymin + 0.88 * (ymax - ymin),
            f"selected: {sel}", fontsize=8)
    ax.set_title(title, fontsize=9)
    ax.set_xlabel("frame index (-)")
    ax.set_ylabel(ylabel)
    ax.grid(True, color="0.85", linewidth=0.5)
fig.tight_layout()
fig_path = os.path.join(FIG_DIR, "exp01_metric_traces.pdf")
fig.savefig(fig_path)
plt.close(fig)
print(f"saved: {fig_path}")

# --- 図 2: 正規化評価値の重ね描き -----
def minmax(arr):
    arr = np.asarray(arr, dtype=np.float64)
    return (arr - arr.min()) / (arr.max() - arr.min())

fig, ax = plt.subplots(figsize=(7.4, 4.0))
styles = [
    (minmax(center_dist), "center-frame  $|i-120|$ ",
     "-", "o", "white"),
    (minmax(dists_s1), "mean-histogram L2 dist. (adopted)",
     "--", "s", "white"),
    (minmax(lap_vars), "Laplacian variance",
     "-.", "^", "black"),
    (minmax(colorfulness), "colorfulness  $MM$ ",
     ":", "v", "black"),
]
for arr, label, ls, marker, mfc in styles:
    ax.plot(frames_axis, arr, linestyle=ls, marker=marker,
            markevery=20, markersize=4, markerfacecolor=mfc,
            color="black", linewidth=0.9, label=label)
ax.set_xlabel("frame index (-)")
ax.set_ylabel("min-max normalized metric value (-)")
ax.grid(True, color="0.85", linewidth=0.5)
ax.legend(fontsize=8, loc="upper left", framealpha=1.0)
fig.tight_layout()
fig_path = os.path.join(FIG_DIR, "exp01_normalized.pdf")
fig.savefig(fig_path)
plt.close(fig)
print(f"saved: {fig_path}")

```

```

# --- 図 3: 各手法の選択フレーム画像 (2x2) -----
cap = cv2.VideoCapture(VIDEO_PATH)
fig, axes = plt.subplots(2, 2, figsize=(8.0, 5.0))
labels = ["(a) Center-frame", "(b) Mean-histogram (adopted)",
          "(c) Sharpness (Laplacian)", "(d) Colorfulness (H-S)"]
for ax, (key, name, sel, t), label in zip(axes.flat, methods, labels):
    cap.set(cv2.CAP_PROP_POS_FRAMES, sel)
    ret, frame = cap.read()
    if not ret:
        raise IOError(f"フレーム {sel} を読み込めません")
    ax.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    ax.set_title(f"{label}: frame {sel}", fontsize=9)
    ax.axis("off")
cap.release()
fig.tight_layout()
fig_path = os.path.join(FIG_DIR, "exp01_selected_frames.pdf")
fig.savefig(fig_path)
plt.close(fig)
print(f"saved: {fig_path}")

print()
print("exp01 done.")

if __name__ == "__main__":
    main()

```

C.2 実験出力: exp01.txt

上記スクリプトの実行出力を無加工で示す。

```

=====
exp01: サムネイル選択手法の定量比較
=====

OpenCV 4.12.0 / NumPy 2.4.4 / matplotlib 3.10.6
video: /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/People - 84973.mp4
size: 1280x720, fps: 25.0, frames(prop): 241
frames(read): 241
decode time total: 4.6410 s

--- 各手法の選択フレームと計算時間 ---
center      中央フレーム法                frame= 120  time=0.000001 s
hist        平均ヒストグラム法 (採用)      frame= 185  time=4.840410 s
laplacian    鮮明度法                      frame= 208  time=6.056869 s
colorfulness カラフルネス法                frame=   8  time=11.725322 s
(参考) 平均ヒストグラム法 sample_step=1: frame=185
(参考) sample_step=5 と step=1 の選択差: 0 フレーム

--- 各評価値の基本統計 (全フレーム) ---
hist_l2_dist(step=1)  min=   0.00706432 max=   0.0206925 mean=   0.0130107 std=   0.00334287
laplacian_var         min=    194.293 max=    230.11 mean=    221.165 std=     5.00327
colorfulness          min=    27.4794 max=    28.2406 mean=    27.9572 std=     0.232764

--- 各手法の選択フレームにおける他指標の値と百分位 ---
(百分位はその値以下のフレームが占める割合 [%]。hist_l2 は小さいほど良い / lap_var, colorfulness は大きいほど良い)

```

method	frame	hist_l2	pct	lap_var	pct	colorful	pct
center	120	0.008142	2.5	218.16	14.9	27.79	30.7
hist	185	0.007064	0.4	221.21	44.8	28.15	78.0
laplacian	208	0.011012	31.5	230.11	100.0	28.17	89.2

```
colorfulness      8    0.018820   92.9    213.01    6.2    28.24  100.0
```

--- 指標間の相関係数 (全フレーム, Pearson / Spearman) ---

```
hist_l2_dist vs laplacian_var    Pearson=-0.3843  Spearman=-0.1613
```

```
hist_l2_dist vs colorfulness    Pearson=+0.3321  Spearman=+0.3143
```

```
laplacian_var vs colorfulness    Pearson=-0.0855  Spearman=+0.0533
```

```
saved: /Users/kimura2003/Downloads/rikadai kadai python/experiments/data/exp01_traces.csv
```

```
saved: /Users/kimura2003/Downloads/rikadai kadai python/experiments/data/exp01_summary.csv
```

```
saved: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp01_metric_traces.pdf
```

```
saved: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp01_normalized.pdf
```

```
saved: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp01_selected_frames.pdf
```

exp01 done.

C.3 実験ソース: exp02_spectrogram.py

スペクトログラムのパラメータ感度実験 (5 章) に用いたスクリプトを無加工で示す。

```
# =====
# exp02: スペクトログラムのパラメータ感度実験
#
# report_audio.wav に対して STFT のパラメータ (窓長 nperseg、窓関数、
# 表示スケール) を系統的に変え、隠されたメッセージ「Good work」の
# 読みやすさを画像コントラスト指標で定量化する。
#
# 実験内容:
# (1) nperseg in {128, 512, 1024, 4096, 16384}(noverlap = 75%、hann)
#     の 5 枚のスペクトログラムをグリッド図 1 枚に描画
# (2) 窓関数 {hann, boxcar, blackman}(nperseg = 1024) の 3 枚比較図
# (3) dB 表示 vs 線形表示の比較図 (nperseg = 1024, hann)
# (4) メッセージ帯域 (1--12 kHz) の表示画素値に対する
#     RMS コントラストとミケルソンコントラスト (P5/P95) を全設定で計算
#
# 出力:
# experiments/figures/exp02_nperseg_grid.pdf
# experiments/figures/exp02_window.pdf
# experiments/figures/exp02_scale.pdf
# experiments/figures/exp02_contrast_vs_nperseg.pdf
# experiments/data/exp02_contrast.csv
# =====

import csv
import os

import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from scipy.io import wavfile

DATASET_ROOT = os.environ.get(
    "DATASET_ROOT", "/content/drive/MyDrive/Colab Notebooks"
)
AUDIO_PATH = os.path.join(DATASET_ROOT, "dataset", "audio", "report_audio.wav")
FIG_DIR = os.path.join(DATASET_ROOT, "experiments", "figures")
DATA_DIR = os.path.join(DATASET_ROOT, "experiments", "data")

# メッセージが目視で確認できる帯域 [Hz] (ベースレポートの図より)
BAND_LOW_HZ = 1000.0
BAND_HIGH_HZ = 12000.0
```



```

# dB 表示のダイナミックレンジ (最大値からの下げ幅) [dB]
DYN_RANGE_DB = 80.0

NPERSEG_LIST = [128, 512, 1024, 4096, 16384]
WINDOW_LIST = ["hann", "boxcar", "blackman"]
BASE_NPERSEG = 1024
OVERLAP_RATIO = 0.75

def load_audio(path):
    """wav を読み込み、モノラルの float64 配列と標本化周波数を返す。"""
    rate, data = wavfile.read(path)
    if data.ndim > 1:
        data = data.mean(axis=1)
    return rate, data.astype(np.float64)

def compute_spectrogram(data, rate, nperseg, window):
    """STFT スペクトログラムを計算する。noverlap は窓長の 75% とする。"""
    noverlap = int(nperseg * OVERLAP_RATIO)
    freqs, times, sxx = signal.spectrogram(
        data, fs=rate, window=window, nperseg=nperseg, noverlap=noverlap
    )
    return freqs, times, sxx

def to_db_image(sxx):
    """パワーを dB 化し、[最大値-DYN_RANGE_DB, 最大値] を [0,1] に正規化する。

    表示 (pcolormesh の vmin/vmax) と同じ規則で正規化した「表示画素値」
    を返すことで、図の見た目とコントラスト指標を対応させる。
    """
    sxx_db = 10.0 * np.log10(sxx + 1e-12)
    vmax = sxx_db.max()
    vmin = vmax - DYN_RANGE_DB
    return np.clip((sxx_db - vmin) / DYN_RANGE_DB, 0.0, 1.0), vmin, vmax

def to_linear_image(sxx):
    """パワーを最大値 1 に正規化した線形表示の画素値を返す。"""
    return sxx / sxx.max()

def band_metrics(image, freqs):
    """メッセージ帯域の表示画素値からコントラスト指標を計算する。

    RMS コントラスト: 帯域内画素値の標準偏差
    ミケルソンコントラスト:  $(P_{95} - P_5) / (P_{95} + P_5)$  (外れ値に頑健な
    5/95 パーセンタイル版)
    """
    mask = (freqs >= BAND_LOW_HZ) & (freqs <= BAND_HIGH_HZ)
    band = image[mask, :]
    p5, p95 = np.percentile(band, [5.0, 95.0])
    rms = float(band.std())
    michelson = float((p95 - p5) / (p95 + p5 + 1e-12))
    return {
        "band_bins": int(mask.sum()),
        "band_cols": int(band.shape[1]),
        "rms_contrast": rms,
        "michelson_p5p95": michelson,
    }

```

```

def draw_panel(ax, times, freqs, values, vmin, vmax, title, cbar_label):
    """1 枚のスペクトログラムをグレースケールで描画する。"""
    mesh = ax.pcolormesh(
        times, freqs / 1000.0, values, shading="auto",
        cmap="gray", vmin=vmin, vmax=vmax, rasterized=True,
    )
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("Frequency [kHz]")
    ax.set_title(title, fontsize=10)
    plt.colorbar(mesh, ax=ax, label=cbar_label)

def main():
    os.makedirs(FIG_DIR, exist_ok=True)
    os.makedirs(DATA_DIR, exist_ok=True)

    rate, data = load_audio(AUDIO_PATH)
    duration = len(data) / rate
    print(f"音声ファイル: {AUDIO_PATH}")
    print(f"標準化周波数: {rate} Hz, サンプル数: {len(data)}, "
          f"長さ: {duration:.2f} 秒")
    print(f"メッセージ帯域: {BAND_LOW_HZ:.0f}--{BAND_HIGH_HZ:.0f} Hz, "
          f"dB 表示ダイナミックレンジ: {DYN_RANGE_DB:.0f} dB")
    print()

    rows = []

    # -----
    # 実験 1: 窓長 nperseg の感度 (hann, dB 表示)
    # -----
    fig, axes = plt.subplots(len(NPERSEG_LIST), 1, figsize=(8.0, 14.0))
    for ax, nperseg in zip(axes, NPERSEG_LIST):
        freqs, times, sxx = compute_spectrogram(data, rate, nperseg, "hann")
        image, vmin, vmax = to_db_image(sxx)
        hop = nperseg - int(nperseg * OVERLAP_RATIO)
        df = rate / nperseg
        dt_ms = hop / rate * 1000.0
        met = band_metrics(image, freqs)
        sxx_db = 10.0 * np.log10(sxx + 1e-12)
        draw_panel(
            ax, times, freqs, sxx_db, vmin, vmax,
            f"nperseg={nperseg} "
            f"($\\Delta f$={df:.1f} Hz, $\\Delta t$={dt_ms:.2f} ms)",
            "Power [dB]",
        )
        rows.append({
            "group": "nperseg", "window": "hann", "scale": "dB",
            "nperseg": nperseg, "noverlap": int(nperseg * OVERLAP_RATIO),
            "hop": hop, "delta_f_hz": round(df, 2),
            "delta_t_ms": round(dt_ms, 3),
            **met,
        })
    fig.tight_layout()
    out = os.path.join(FIG_DIR, "exp02_nperseg_grid.pdf")
    fig.savefig(out)
    plt.close(fig)
    print(f"図を保存しました: {out}")

    # -----

```

```

# 実験 2: 窓関数の感度 (nperseg=1024、dB 表示)
# -----
fig, axes = plt.subplots(len(WINDOW_LIST), 1, figsize=(8.0, 9.0))
for ax, window in zip(axes, WINDOW_LIST):
    freqs, times, sxx = compute_spectrogram(
        data, rate, BASE_NPERSEG, window
    )
    image, vmin, vmax = to_db_image(sxx)
    hop = BASE_NPERSEG - int(BASE_NPERSEG * OVERLAP_RATIO)
    met = band_metrics(image, freqs)
    sxx_db = 10.0 * np.log10(sxx + 1e-12)
    draw_panel(
        ax, times, freqs, sxx_db, vmin, vmax,
        f"window={window} (nperseg={BASE_NPERSEG})", "Power [dB]",
    )
    rows.append({
        "group": "window", "window": window, "scale": "dB",
        "nperseg": BASE_NPERSEG,
        "noverlap": int(BASE_NPERSEG * OVERLAP_RATIO),
        "hop": hop, "delta_f_hz": round(rate / BASE_NPERSEG, 2),
        "delta_t_ms": round(hop / rate * 1000.0, 3),
        **met,
    })
fig.tight_layout()
out = os.path.join(FIG_DIR, "exp02_window.pdf")
fig.savefig(out)
plt.close(fig)
print(f"図を保存しました: {out}")

# -----
# 実験 3: dB 表示 vs 線形表示 (nperseg=1024、hann)
# -----
freqs, times, sxx = compute_spectrogram(data, rate, BASE_NPERSEG, "hann")
hop = BASE_NPERSEG - int(BASE_NPERSEG * OVERLAP_RATIO)
fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.5))

image_db, vmin, vmax = to_db_image(sxx)
sxx_db = 10.0 * np.log10(sxx + 1e-12)
draw_panel(
    axes[0], times, freqs, sxx_db, vmin, vmax,
    "dB scale (nperseg=1024, hann)", "Power [dB]",
)
met_db = band_metrics(image_db, freqs)
rows.append({
    "group": "scale", "window": "hann", "scale": "dB",
    "nperseg": BASE_NPERSEG, "noverlap": int(BASE_NPERSEG * OVERLAP_RATIO),
    "hop": hop, "delta_f_hz": round(rate / BASE_NPERSEG, 2),
    "delta_t_ms": round(hop / rate * 1000.0, 3),
    **met_db,
})

image_lin = to_linear_image(sxx)
draw_panel(
    axes[1], times, freqs, image_lin, 0.0, 1.0,
    "linear scale (nperseg=1024, hann)", "Normalized power",
)
met_lin = band_metrics(image_lin, freqs)
rows.append({
    "group": "scale", "window": "hann", "scale": "linear",
    "nperseg": BASE_NPERSEG, "noverlap": int(BASE_NPERSEG * OVERLAP_RATIO),
    "hop": hop, "delta_f_hz": round(rate / BASE_NPERSEG, 2),
})

```

```

        "delta_t_ms": round(hop / rate * 1000.0, 3),
        **met_lin,
    })
    fig.tight_layout()
    out = os.path.join(FIG_DIR, "exp02_scale.pdf")
    fig.savefig(out)
    plt.close(fig)
    print(f"図を保存しました: {out}")

# -----
# 実験 4: コントラスト指標 vs 窓長のプロット (白黒判別可能)
# -----
nperseg_rows = [r for r in rows if r["group"] == "nperseg"]
xs = [r["nperseg"] for r in nperseg_rows]
rms_vals = [r["rms_contrast"] for r in nperseg_rows]
mic_vals = [r["michelson_p5p95"] for r in nperseg_rows]
fig, ax = plt.subplots(figsize=(7.0, 4.5))
ax.plot(xs, rms_vals, "k-o", label="RMS contrast")
ax.plot(xs, mic_vals, "k--s", label="Michelson contrast (P5/P95)")
ax.set_xscale("log", base=2)
ax.set_xticks(xs)
ax.set_xticklabels([str(x) for x in xs])
ax.set_xlabel("nperseg [samples]")
ax.set_ylabel("Contrast index")
ax.grid(True, linestyle=":")
ax.legend()
fig.tight_layout()
out = os.path.join(FIG_DIR, "exp02_contrast_vs_nperseg.pdf")
fig.savefig(out)
plt.close(fig)
print(f"図を保存しました: {out}")

# -----
# CSV 保存と結果表示
# -----
csv_path = os.path.join(DATA_DIR, "exp02_contrast.csv")
fieldnames = [
    "group", "window", "scale", "nperseg", "noverlap", "hop",
    "delta_f_hz", "delta_t_ms",
    "band_bins", "band_cols", "rms_contrast", "michelson_p5p95",
]
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for row in rows:
        writer.writerow(row)
print(f"CSV を保存しました: {csv_path}")
print()

header = (f"{'group':<8} {'window':<9} {'scale':<7} {'nperseg':>7} "
          f"{'df[Hz]':>8} {'dt[ms]':>8} "
          f"{'bins':>5} {'cols':>5} {'RMS':>7} {'Michelson':>9}")
print(header)
print("-" * len(header))
for r in rows:
    print(f"{r['group']:<8} {r['window']:<9} {r['scale']:<7} "
          f"{r['nperseg']:>7} {r['delta_f_hz']:>8.2f} "
          f"{r['delta_t_ms']:>8.3f} "
          f"{r['band_bins']:>5} {r['band_cols']:>5} "
          f"{r['rms_contrast']:>7.4f} {r['michelson_p5p95']:>9.4f}")

```

```
if __name__ == "__main__":
    main()
```

C.4 実験出力: exp02.txt

上記スクリプトの実行出力を無加工で示す。

音声ファイル: /Users/kimura2003/Downloads/rikadai kadai python/dataset/audio/report_audio.wav
 標本化周波数: 44100 Hz, サンプル数: 264600, 長さ: 6.00 秒
 メッセージ帯域: 1000-12000 Hz, dB 表示ダイナミックレンジ: 80 dB

図を保存しました: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp02_nperseg_grid.pdf
 図を保存しました: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp02_window.pdf
 図を保存しました: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp02_scale.pdf
 図を保存しました: /Users/kimura2003/Downloads/rikadai kadai python/experiments/figures/exp02_contrast_vs_nperseg.pdf
 CSV を保存しました: /Users/kimura2003/Downloads/rikadai kadai python/experiments/data/exp02_contrast.csv

group	window	scale	nperseg	df [Hz]	dt [ms]	bins	cols	RMS Michelson	
nperseg	hann	dB	128	344.53	0.726	32	8265	0.2995	0.9292
nperseg	hann	dB	512	86.13	2.902	128	2064	0.2966	1.0000
nperseg	hann	dB	1024	43.07	5.805	255	1030	0.2719	1.0000
nperseg	hann	dB	4096	10.77	23.220	1022	255	0.2422	1.0000
nperseg	hann	dB	16384	2.69	92.880	4087	61	0.1961	1.0000
window	hann	dB	1024	43.07	5.805	255	1030	0.2719	1.0000
window	boxcar	dB	1024	43.07	5.805	255	1030	0.1617	0.4101
window	blackman	dB	1024	43.07	5.805	255	1030	0.3063	1.0000
scale	hann	dB	1024	43.07	5.805	255	1030	0.2719	1.0000
scale	hann	linear	1024	43.07	5.805	255	1030	0.2230	1.0000

C.5 実験ソース: exp03_invert_timing.py

```
# -*- coding: utf-8 -*-
"""exp03: 色反転処理の実装方式と実行時間のスケーリング計測
```

課題 4 で採用した `numpy` ベクトル演算 (`255 - frame`) による色反転について、
 実装方式ごとの実行時間と、画素数に対するスケーリングを計測する。

計測する実装方式 (1 フレーム 1280x720x3 uint8):

- (a) `numpy` ベクトル演算 `255 - frame` … 提出プログラムの採用手法
- (b) `np.subtract(255, frame, out=buf) in-place` … 出力バッファ再利用版
- (c) Python 3 重ループ … 160x90 のみ実測し、
画素数比でフルサイズへ換算
- (d) `cv2.bitwise_not(frame)` … 参考計測。課題の制約で
提出プログラムでは使用禁止

さらに、

- 解像度 {160x90, 320x180, 640x360, 1280x720, 2560x1440} で
(a)(d) の時間を計測し、`log-log` の傾きを最小二乗で推定する。
- 動画全体 (294 フレーム) の処理時間を
読み込み / 反転 / リサイズ / 書き出し の 4 区間に分けて計測する。

実行方法:

```
cd "/Users/kimura2003/Downloads/rikadai kadai python" && \
  DATASET_ROOT="$PWD" MPLBACKEND=Agg \
  python3 experiments/exp03_invert_timing.py > results/exp03.txt 2>&1
"""
```

```

import csv
import os
import platform
import statistics
import time
import timeit

import cv2
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

# -----
# パス設定
# -----
ROOT = os.environ.get("DATASET_ROOT", ".")
VIDEO_PATH = os.path.join(ROOT, "dataset", "videos", "Cat - 66004.mp4")
DATA_DIR = os.path.join(ROOT, "experiments", "data")
FIG_DIR = os.path.join(ROOT, "experiments", "figures")
OUT_VIDEO = os.path.join(ROOT, "dataset", "videos", "output",
                          "exp03_invert_0.3x.mp4")
os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(os.path.dirname(OUT_VIDEO), exist_ok=True)

# 計測条件
REPEAT = 7          # timeit.repeat の繰り返し回数 (中央値を採用)
TARGET_TOTAL = 0.20 # 1 回の repeat の目標総時間 [s] (number を自動決定)
MIN_NUMBER = 5       # 1 repeat あたりの最小実行回数 (高速な処理の安定化)
MAX_NUMBER = 2000
WARMUP_CALLS = 3     # 計測前の空実行回数 (キャッシュ・遅延初期化の除去)
LOOP_SIZE = (160, 90) # Python 3 重ループの実測サイズ (width, height)
SCALES = [(160, 90), (320, 180), (640, 360), (1280, 720), (2560, 1440)]
RESIZE_SCALE = 0.3   # 課題 4 と同じ縮小率

# 白黒印刷で判別できる描画設定
matplotlib.rcParams.update({
    "font.size": 11,
    "axes.grid": True,
    "grid.linestyle": ":",
    "grid.color": "0.7",
    "figure.dpi": 150,
})

# -----
# 色反転の各実装
# -----
def invert_vec(frame):
    """(a) numpy ベクトル演算 (提出プログラムの採用手法)。"""
    return 255 - frame

def invert_inplace(frame, buf):
    """(b) 出力バッファを再利用する in-place 版。"""
    np.subtract(255, frame, out=buf)
    return buf

def invert_loop(frame):
    """(c) Python 3 重ループ (画素ごとに 255 - p を計算)。"""

```

```

    h, w, c = frame.shape
    out = np.empty_like(frame)
    for y in range(h):
        for x in range(w):
            for ch in range(c):
                out[y, x, ch] = 255 - frame[y, x, ch]
    return out

def invert_cv2(frame):
    """(d) cv2.bitwise_not(参考計測。提出プログラムでは使用禁止)。"""
    return cv2.bitwise_not(frame)

# -----
# 計測ユーティリティ
# -----
def measure(fn, target_total=TARGET_TOTAL, repeat=REPEAT, min_number=MIN_NUMBER):
    """fn の 1 回あたり実行時間を timeit.repeat で計測する。

    計測前に WARMUP_CALLS 回の空実行を行い (キャッシュ・遅延初期化の影響を
    除去)、その後の 3 回の単発計測の最小値から、1 repeat の総時間が
    target_total 程度になるよう number を自動決定する (min_number 以上)。
    repeat 回の per-call 時間の中央値・最小値・最大値を返す。
    """
    for _ in range(WARMUP_CALLS):
        fn()
    timer = timeit.Timer(fn)
    cal = min(timer.timeit(number=1) for _ in range(3)) # キャリブレーション
    number = int(max(min_number,
                     min(MAX_NUMBER, round(target_total / max(cal, 1e-9)))))
    raw = timer.repeat(repeat=repeat, number=number)
    per_call = sorted(t / number for t in raw)
    return {
        "median_s": statistics.median(per_call),
        "min_s": per_call[0],
        "max_s": per_call[-1],
        "number": number,
        "repeat": repeat,
    }

def fmt_ms(sec):
    return f"{sec * 1e3:.4f}"

# -----
# 準備: フレームの取得
# -----
print("=" * 70)
print("exp03: 色反転処理の実装方式と実行時間のスケーリング計測")
print("=" * 70)
print(f"platform : {platform.platform()}")
print(f"machine   : {platform.machine()}")
print(f"python    : {platform.python_version()}")
print(f"numpy     : {np.__version__}")
print(f"opencv    : {cv2.__version__}")
print(f"video     : {VIDEO_PATH}")
print(f"timeit    : repeat={REPEAT}, 1 repeat の目標総時間 {TARGET_TOTAL} s "
      f"(number は自動決定), 統計量は中央値")
print()

```

```

cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise IOError(f"動画を開けません: {VIDEO_PATH}")
ret, frame_full = cap.read()
if not ret:
    raise IOError("先頭フレームを読み込めません")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap.get(cv2.CAP_PROP_FPS)
cap.release()

h, w, c = frame_full.shape
n_pixels_full = h * w
print(f"先頭フレーム: {w}x{h}x{c} dtype={frame_full.dtype}, "
      f"総フレーム数 {n_frames}, fps {fps:.2f}")
print()

# -----
# 正しさの確認 (4 方式が同一の結果を返すこと)
# -----
print("-" * 70)
print("[0] 各実装の同値性確認")
print("-" * 70)
ref = invert_vec(frame_full)
buf = np.empty_like(frame_full)
same_b = np.array_equal(ref, invert_inplace(frame_full, buf))
same_d = np.array_equal(ref, invert_cv2(frame_full))
frame_small = cv2.resize(frame_full, LOOP_SIZE)
same_c = np.array_equal(invert_vec(frame_small), invert_loop(frame_small))
print(f"(b) in-place == (a) ベクトル演算 : {same_b}")
print(f"(c) 3 重ループ == (a) ベクトル演算 : {same_c} ({LOOP_SIZE[0]}x{LOOP_SIZE[1]} で確認)")
print(f"(d) bitwise_not == (a) ベクトル演算 : {same_d}")
assert same_b and same_c and same_d
print()

# -----
# [1] 1 フレーム (1280x720) に対する 4 方式の実行時間
# -----
print("-" * 70)
print("[1] 1 フレーム (1280x720x3 uint8) に対する 4 方式の実行時間")
print("-" * 70)

results_methods = []

r = measure(lambda: invert_vec(frame_full))
results_methods.append(("a_vec", "255 - frame (採用手法)", w, h, r, "measured"))

r = measure(lambda: invert_inplace(frame_full, buf))
results_methods.append(("b_inplace", "np.subtract(out=) in-place", w, h, r,
                        "measured"))

# (c) Python 3 重ループは 160x90 で実測し、画素数比でフルサイズへ換算
r_loop = measure(lambda: invert_loop(frame_small), target_total=0.0,
                  min_number=1)
sw, sh = LOOP_SIZE
scale_factor = (w * h) / (sw * sh)
results_methods.append(("c_loop_small", "Python 3 重ループ (160x90 実測)",
                        sw, sh, r_loop, "measured"))

r_loop_est = {
    "median_s": r_loop["median_s"] * scale_factor,
    "min_s": r_loop["min_s"] * scale_factor,

```



```

        "max_s": r_loop["max_s"] * scale_factor,
        "number": r_loop["number"],
        "repeat": r_loop["repeat"],
    }
    results_methods.append(("c_loop_est", "Python 3 重ループ (1280x720 換算値)",
                           w, h, r_loop_est, f"extrapolated x{scale_factor:.0f}"))

r = measure(lambda: invert_cv2(frame_full))
results_methods.append(("d_cv2", "cv2.bitwise_not (参考・提出では使用禁止)",
                       w, h, r, "measured"))

print(f"{'方式':<40s} {'サイズ':>10s} {'中央値 [ms]':>12s} "
      f"{'最小 [ms]':>10s} {'最大 [ms]':>10s} {'number':>7s} {'備考':>16s}")
for key, label, mw, mh, r, note in results_methods:
    print(f"{label:<40s} {mw:>5d}x{mh:<4d} {fmt_ms(r['median_s']):>12s} "
          f"{fmt_ms(r['min_s']):>10s} {fmt_ms(r['max_s']):>10s} "
          f"{r['number']:>7d} {note:>16s}")
print()

med = {key: r["median_s"] for key, _, _, _, r, _ in results_methods}
ratio_loop_vec = med["c_loop_est"] / med["a_vec"]
ratio_vec_cv2 = med["a_vec"] / med["d_cv2"]
ratio_vec_inplace = med["a_vec"] / med["b_inplace"]
print(f"倍率: 3 重ループ (換算) / ベクトル演算 (a)    = {ratio_loop_vec:.1f} 倍")
print(f"倍率: ベクトル演算 (a) / in-place(b)        = {ratio_vec_inplace:.2f} 倍")
print(f"倍率: ベクトル演算 (a) / bitwise_not(d)      = {ratio_vec_cv2:.2f} 倍")
print()

csv_methods = os.path.join(DATA_DIR, "exp03_methods.csv")
with open(csv_methods, "w", newline="") as f:
    wcsv = csv.writer(f)
    wcsv.writerow(["key", "label", "width", "height", "pixels",
                   "median_ms", "min_ms", "max_ms", "number", "repeat",
                   "note"])
    for key, label, mw, mh, r, note in results_methods:
        wcsv.writerow([key, label, mw, mh, mw * mh,
                       f"{r['median_s'] * 1e3:.6f}",
                       f"{r['min_s'] * 1e3:.6f}",
                       f"{r['max_s'] * 1e3:.6f}",
                       r["number"], r["repeat"], note])
print(f"CSV 保存: {csv_methods}")

# 図 1: 方式別実行時間 (横棒、対数軸、白黒判別: グレー濃淡 + ハッチ)
fig, ax = plt.subplots(figsize=(7.2, 3.4))
bar_items = [
    ("(a) 255 - frame", med["a_vec"] * 1e3, "0.35", ""),
    ("(b) np.subtract(out=)", med["b_inplace"] * 1e3, "0.55", ""),
    ("(c) Python triple loop\n(extrapolated)", med["c_loop_est"] * 1e3,
     "0.85", "//"),
    ("(d) cv2.bitwise_not\n(reference)", med["d_cv2"] * 1e3, "0.7", "xx"),
]
labels = [b[0] for b in bar_items]
values = [b[1] for b in bar_items]
colors = [b[2] for b in bar_items]
hatches = [b[3] for b in bar_items]
bars = ax.barh(range(len(bar_items)), values, color=colors,
               edgecolor="black", height=0.6)
for bar, hatch in zip(bars, hatches):
    bar.set_hatch(hatch)
for i, v in enumerate(values):
    ax.text(v * 1.15, i, f"{v:.3g} ms", va="center", fontsize=10)

```

```

ax.set_yticks(range(len(bar_items)))
ax.set_yticklabels(labels)
ax.set_xscale("log")
ax.set_xlim(right=max(values) * 8)
ax.set_xlabel("Time per frame [ms] (median, 1280x720x3 uint8)")
ax.invert_yaxis()
fig.tight_layout()
fig_methods = os.path.join(FIG_DIR, "exp03_methods.pdf")
fig.savefig(fig_methods)
plt.close(fig)
print(f"図 保存: {fig_methods}")
print()

# -----
# [2] 解像度スケーリング: (a) と (d) の時間 vs 画素数
# -----

print("-" * 70)
print("[2] 解像度スケーリング ((a) ベクトル演算 と (d) bitwise_not)")
print("-" * 70)

scaling_rows = []
print(f"{' 解像度':>12s} {' 画素数':>10s} {'(a) 中央値 [ms]':>14s} "
      f"{'(d) 中央値 [ms]':>14s}")
for sw_, sh_ in SCALES:
    if (sw_, sh_) == (w, h):
        fr = frame_full
    else:
        interp = cv2.INTER_AREA if sw_ < w else cv2.INTER_LINEAR
        fr = cv2.resize(frame_full, (sw_, sh_), interpolation=interp)
    np_ = sw_ * sh_
    ra = measure(lambda: invert_vec(fr))
    rd = measure(lambda: invert_cv2(fr))
    scaling_rows.append((sw_, sh_, np_, ra, rd))
    print(f"{sw_:>6d}x{sh_:<5d} {np_:>10d} {fmt_ms(ra['median_s']):>14s} "
          f"{fmt_ms(rd['median_s']):>14s}")

# log-log の傾きを最小二乗で推定
px = np.array([row[2] for row in scaling_rows], dtype=np.float64)
ta = np.array([row[3]["median_s"] for row in scaling_rows])
td = np.array([row[4]["median_s"] for row in scaling_rows])
slope_a, icpt_a = np.polyfit(np.log10(px), np.log10(ta), 1)
slope_d, icpt_d = np.polyfit(np.log10(px), np.log10(td), 1)
print()
print(f"log-log 傾き (a) 255 - frame      : {slope_a:.3f}")
print(f"log-log 傾き (d) cv2.bitwise_not : {slope_d:.3f}")
print()

csv_scaling = os.path.join(DATA_DIR, "exp03_scaling.csv")
with open(csv_scaling, "w", newline="") as f:
    wcsv = csv.writer(f)
    wcsv.writerow(["width", "height", "pixels",
                   "a_vec_median_ms", "a_vec_min_ms", "a_vec_number",
                   "d_cv2_median_ms", "d_cv2_min_ms", "d_cv2_number"])
    for sw_, sh_, np_, ra, rd in scaling_rows:
        wcsv.writerow([sw_, sh_, np_,
                       f"{ra['median_s']} * 1e3:.6f",
                       f"{ra['min_s']} * 1e3:.6f", ra["number"],
                       f"{rd['median_s']} * 1e3:.6f",
                       f"{rd['min_s']} * 1e3:.6f", rd["number"]])
print(f"CSV 保存: {csv_scaling}")

```

```

# 図 2: log-log プロット (白黒判別: 実線+丸、破線+四角、点線=傾き 1 基準線)
fig, ax = plt.subplots(figsize=(6.4, 4.6))
ax.loglog(px, ta * 1e3, "o-", color="black",
          label=f"(a) 255 - frame (slope {slope_a:.2f})")
ax.loglog(px, td * 1e3, "s--", color="0.45",
          label=f"(d) cv2.bitwise_not (slope {slope_d:.2f})")
# 傾き 1 の基準線 ((a) の最終点を通る)
ref_y = ta[-1] * 1e3 * (px / px[-1])
ax.loglog(px, ref_y, ":", color="0.6", label="slope = 1 (reference)")
ax.set_xlabel("Number of pixels per frame")
ax.set_ylabel("Time per frame [ms] (median)")
ax.legend(fontsize=9)
fig.tight_layout()
fig_scaling = os.path.join(FIG_DIR, "exp03_scaling.pdf")
fig.savefig(fig_scaling)
plt.close(fig)
print(f"図 保存: {fig_scaling}")
print()

# -----
# [3] 動画全体 (294 フレーム) の処理時間内訳
# -----
print("-" * 70)
print("[3] 動画全体の処理時間内訳 (読み込み / 反転 / リサイズ / 書き出し)")
print("-" * 70)

cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise IOError(f"動画を開けません: {VIDEO_PATH}")
vw = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
vh = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
vfps = cap.get(cv2.CAP_PROP_FPS)
new_size = (int(vw * RESIZE_SCALE), int(vh * RESIZE_SCALE))
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(OUT_VIDEO, fourcc, vfps, new_size)

t_read = t_invert = t_resize = t_write = 0.0
frame_count = 0
t_total0 = time.perf_counter()
while True:
    t0 = time.perf_counter()
    ret, fr = cap.read()
    t1 = time.perf_counter()
    t_read += t1 - t0
    if not ret:
        break
    inv = 255 - fr
    t2 = time.perf_counter()
    t_invert += t2 - t1
    rs = cv2.resize(inv, new_size)
    t3 = time.perf_counter()
    t_resize += t3 - t2
    writer.write(rs)
    t4 = time.perf_counter()
    t_write += t4 - t3
    frame_count += 1
cap.release()
writer.release()
t_total = time.perf_counter() - t_total0

stages = [

```

```

        ("read", "読み込み (cap.read)", t_read),
        ("invert", "反転 (255 - frame)", t_invert),
        ("resize", "リサイズ (cv2.resize)", t_resize),
        ("write", "書き出し (writer.write)", t_write),
    ]
    t_stages = sum(s[2] for s in stages)
    print(f"入力: {vw}x{vh}, {frame_count} フレーム, fps {vfps:.2f}")
    print(f"出力: {new_size[0]}x{new_size[1]}, {OUT_VIDEO}")
    print()
    print(f"{' 区間':<28s} {' 合計 [s]':>10s} {'1 フレーム平均 [ms]':>18s} {' 割合 [%]':>9s}")
    for key, label, t in stages:
        print(f"{label:<28s} {t:>10.4f} {t / frame_count * 1e3:>18.3f} "
              f"{t / t_stages * 100:>9.1f}")
    print(f"{'4 区間合計':<28s} {t_stages:>10.4f} "
          f"{t_stages / frame_count * 1e3:>18.3f} {100.0:>9.1f}")
    print(f"{' 全体 (perf_counter)':<28s} {t_total:>10.4f} "
          f"{t_total / frame_count * 1e3:>18.3f}")
    print()

    csv_pipeline = os.path.join(DATA_DIR, "exp03_pipeline.csv")
    with open(csv_pipeline, "w", newline="") as f:
        wcsv = csv.writer(f)
        wcsv.writerow(["stage", "label", "total_s", "per_frame_ms",
                       "share_percent", "frames"])
        for key, label, t in stages:
            wcsv.writerow([key, label, f"{t:.6f}",
                           f"{t / frame_count * 1e3:.6f}",
                           f"{t / t_stages * 100:.3f}", frame_count])
        wcsv.writerow(["total_stages", "4 区間合計", f"{t_stages:.6f}",
                       f"{t_stages / frame_count * 1e3:.6f}", "100.000",
                       frame_count])
        wcsv.writerow(["total_wall", "全体", f"{t_total:.6f}",
                       f"{t_total / frame_count * 1e3:.6f}", "", frame_count])
    print(f"CSV 保存: {csv_pipeline}")

# 図 3: 内訳の積み上げ横棒 (白黒判別: グレー濃淡 + ハッチ)
fig, ax = plt.subplots(figsize=(7.2, 2.6))
stage_style = [
    ("read (decode)", t_read, "0.85", ""),
    ("invert (255 - frame)", t_invert, "0.35", ""),
    ("resize", t_resize, "0.6", "/"),
    ("write (encode)", t_write, "0.95", "xx"),
]
left = 0.0
for label, t, color, hatch in stage_style:
    ax.barh([0], [t], left=left, color=color, edgecolor="black",
            hatch=hatch, height=0.5, label=label)
    if t / t_stages > 0.04:
        ax.text(left + t / 2, 0, f"{t:.2f} s\n({t / t_stages * 100:.1f}%)",
                ha="center", va="center", fontsize=9,
                bbox=dict(facecolor="white", edgecolor="none", alpha=0.85,
                        pad=1.5))
    left += t
ax.set_yticks([])
ax.set_xlabel(f"Cumulative time for {frame_count} frames [s]")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.35), ncol=4,
        fontsize=9, frameon=False)
ax.set_xlim(0, t_stages * 1.02)
fig.tight_layout()
fig_pipeline = os.path.join(FIG_DIR, "exp03_pipeline.pdf")
fig.savefig(fig_pipeline, bbox_inches="tight")

```

```
plt.close(fig)
print(f"図 保存: {fig_pipeline}")
print()

print("=" * 70)
print("exp03 完了")
print("=" * 70)
```

C.6 実験出力: exp03.txt

=====

exp03: 色反転処理の実装方式と実行時間のスケーリング計測

=====

```
platform : macOS-26.5-arm64-arm-64bit-Mach-0
machine  : arm64
python   : 3.13.2
numpy    : 2.4.4
opencv   : 4.12.0
video    : /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/Cat - 66004.mp4
timeit    : repeat=7, 1 repeat の目標総時間 0.2 s (number は自動決定), 統計量は中央値
```

先頭フレーム: 1280x720x3 dtype=uint8, 総フレーム数 294, fps 29.97

[0] 各実装の同値性確認

```
(b) in-place == (a) ベクトル演算 : True
(c) 3 重ループ == (a) ベクトル演算 : True (160x90 で確認)
(d) bitwise_not == (a) ベクトル演算 : True
```

[1] 1 フレーム (1280x720x3 uint8) に対する 4 方式の実行時間

方式	サイズ	中央値 [ms]	最小 [ms]	最大 [ms]	number	
255 - frame (採用手法)	1280x720	2.2157	0.6907	2.5249	1880	measured
np.subtract(out=) in-place	1280x720	0.7896	0.2959	1.4665	1236	measured
Python 3 重ループ (160x90 実測)	160x90	39.9825	20.9705	87.6172	1	measured
Python 3 重ループ (1280x720 換算値)	1280x720	2558.8826	1342.1120	5607.4986	1	extrapolated
cv2.bitwise_not (参考・提出では使用禁止)	1280x720	1.3317	0.8164	3.2829	1966	measured

倍率: 3 重ループ (換算) / ベクトル演算 (a) = 1154.9 倍
 倍率: ベクトル演算 (a) / in-place(b) = 2.81 倍
 倍率: ベクトル演算 (a) / bitwise_not(d) = 1.66 倍

CSV 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/data/exp03_methods.csv
 図 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/figures/exp03_methods.pdf

[2] 解像度スケーリング ((a) ベクトル演算 と (d) bitwise_not)

解像度	画素数	(a) 中央値 [ms]	(d) 中央値 [ms]
160x90	14400	0.0210	0.0174
320x180	57600	0.0793	0.0852
640x360	230400	0.2502	0.4770
1280x720	921600	0.6909	1.8703
2560x1440	3686400	7.4146	4.0265

log-log 傾き (a) 255 - frame : 1.002
 log-log 傾き (d) cv2.bitwise_not : 1.009

CSV 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/data/exp03_scaling.csv
図 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/figures/exp03_scaling.pdf

[3] 動画全体の処理時間内訳 (読み込み / 反転 / リサイズ / 書き出し)

入力: 1280x720, 294 フレーム, fps 29.97

出力: 384x216, /Users/kimura2003/Downloads/rikadai_kadai_python/dataset/videos/output/exp03_invert_0.3x.mp4

区間	合計 [s]	1 フレーム平均 [ms]	割合 [%]
読み込み (cap.read)	5.2918	17.999	67.7
反転 (255 - frame)	0.4771	1.623	6.1
リサイズ (cv2.resize)	0.8733	2.971	11.2
書き出し (writer.write)	1.1724	3.988	15.0
4 区間合計	7.8146	26.580	100.0
全体 (perf_counter)	7.9817	27.149	

CSV 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/data/exp03_pipeline.csv

図 保存: /Users/kimura2003/Downloads/rikadai_kadai_python/experiments/figures/exp03_pipeline.pdf

=====
exp03 完了
=====